DAY 1

Precision and width

Given 2 numbers, print the rst number as a 5-digit number and print the second number in the next line with 5 width space

Input Format

int int

Constraints

NULL

Output Format

INT INT

Sample Input 0

25 98

Sample Output 0

00025

  98

```
import java.io.*;
import java.util.*;

public class Solution {

   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int a=sc.nextInt();
      int b=sc.nextInt();
      System.out.printf("%05d\n",a);
      System.out.printf("%5d",b);
   }
```

}

Value Performance 2

Kumar wants to know how to display the numbers in programming.can you please guide him to display the numbers.

Input Format

First input corresponds to the Integer Number. Second input corresponds to the Float Number.

Constraints

No Constraints

Output Format

First output corresponds to the Integer Number Second output corresponds to the Float Number

Sample Input 0

4

5.678

Sample Output 0

4

5.68

```
import java.io.*;

import java.util.*;


public class Solution {


  public static void main(String[] args) {

    Scanner sc=new Scanner(System.in);

    int a=sc.nextInt();

    float b=sc.nextFloat();

    System.out.println(a);
```

```
    System.out.format("%.2f",b);
  }}
```

Float Formation 2

Alexa wants to know how the decimal digits are working in programming.can you please guide her to working on decimal digit.

Input Format

Input corresponds to the Float Value.

Constraints

No Constraints

Output Format

Output Corresponds to the Decimal Digit values.

Sample Input 0

67.98

Sample Output 0

67.980003

67.9800

67.98

68

```
import java.io.*;
import java.util.*;
public class Solution {
  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    float n=sc.nextFloat();
    System.out.printf("%.6f\n",n);
    System.out.printf("%.4f\n",n);
    System.out.printf("%.2f\n",n);
    System.out.printf("%.0f",n);
```

```
  }
}
```

Math Functions 2

Sila wants to know how to calculate the values using math function.can you please guide her to perform the math Calculations. Note:use the Floor, Ceil, Sqrt, Pow

Input Format

First Input corresponds to the float value Second input correspond to the Integer value is for square value. Third input correspond to the Integer value is for base value. Fourth input correspond to the Integer value is for power value.

Constraints

No Constraints

Output Format

Output corressponds to the calculation of Math functions.

Sample Input 0

76.78

16

2

3

```
import java.io.*;
import java.util.*;

public class Solution {

  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    float a=sc.nextFloat();
    int b=sc.nextInt();
    int base=sc.nextInt();
    int pov=sc.nextInt();
```

```
        System.out.println((int)Math.floor(a));
        System.out.println((int)Math.ceil(a));
        System.out.println((int)Math.sqrt(b));
        System.out.println((int)Math.pow(base,pov));
    }
}
```

ATM machine

Write a C program to input amount from user and print minimum number of notes (Rs.500,100,50,20,10,5,2,1) required for the given amount in c

Input Format

Integer

Constraints

1 < n < 1000000

Output Format

Integer

Sample Input 0

750

Sample Output 0

Total number of notes:

500 : 1

100 : 2

50 : 1

20 : 0

10 : 0

5 : 0

2 : 0

1 : 0

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
  public static void main(String args[] ) throws Exception {
    Scanner sc=new Scanner(System.in);
    int num=sc.nextInt();
    System.out.println("Total number of notes:");
    if(num>=500){
      int x= (int)Math.floor(num/500);
      System.out.println("500 : "+x);
      num %=500;
    }else{
      System.out.println("500 : 0");
    }
    if(num>=100){
      int y= (int)Math.floor(num/100);
      System.out.println("100 : "+y);
      num %=100;
    }else{
      System.out.println("100 : 0");
    }
    if(num>=50){
      int z= (int)Math.floor(num/50);
      System.out.println("50 : "+z);
```

```
    num %=50;
}else{
    System.out.println("50 : 0");
}if(num>=20){
    int k= (int)Math.floor(num/20);
    System.out.println("20 : "+k);
    num %=20;
}else{
    System.out.println("20 : 0");
}if(num>=10){
    int j= (int)Math.floor(num/10);
    System.out.println("10 : "+j);
    num %=10;
}else{
    System.out.println("10 : 0");
}if(num>=5){
    int l= (int)Math.floor(num/5);
    System.out.println("5 : "+l);
    num %=5;
}else{
    System.out.println("5 : 0");
}if(num>=2){
    int s= (int)Math.floor(num/2);
    System.out.println("2 : "+s);
    num %=2;
}else{
    System.out.println("2 : 0");
}if(num>=1){
```

```
        int m= (int)Math.floor(num/1);
        System.out.println("1 : "+m);
        num %=1;

      }else{
        System.out.println("1 : 0");
      }
    }
}
```

Execute the word with a Statement. 1

Anil wants to know how to display the word with the statement in the program.can you please guide him to display the Statement.

Input Format

First input corresponds to the String.

Constraints

No Constraints

Output Format

Output corresponds to display the string with Statement.

Sample Input 0

C programming

Sample Output 0

May I know how to learn C!!!...

```
import java.io.*;
import java.util.*;

public class Solution {
```

```
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String str=sc.next();
        System.out.println("May I know how to learn "+str+"!!!...");
    }
}
```

C-L0-MOD2-Conditional-Challenge-1

Write a C program to check the values are equal or not

Input Format

Accept two integer as input

Constraints

NIL

Output Format

EQUAL/NOT EQUAL

Sample Input 0

10 10

Sample Output 0

EQUAL

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
```

```
  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int a=sc.nextInt();
    int b=sc.nextInt();
    if(a==b){
      System.out.println("EQUAL");
    }else{
      System.out.println("NOT EQUAL");
    }
  }
}
```

C-L0-MOD2-Conditional-Challenge-2

Write a C program to check the number is odd or even using conditional construct

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

ODD/EVEN

Sample Input 0

3

Sample Output 0

ODD

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
```

```
import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    if(n%2==0){
      System.out.println("EVEN");
    }else{
      System.out.println("ODD");
    }
  }
}
```

C-L0-MOD2-Conditional-Challenge-3

Write a C program to determine the sign of the input using conditional constructs.

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

Positive/Negative/Zero

Sample Input 0

100

Sample Output 0

Positive

```
import java.io.*;
```

```
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int a=sc.nextInt();
    if(a==0){
      System.out.println("Zero");
    }else if(a>0){
      System.out.println("Positive");
    }else{
      System.out.println("Negative");
    }
  }
}
```

C-L0-MOD2-Conditional-Challenge-4

Write a C program to check the number is divisible by 3 or not using conditional construct.

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

DIVISIBLE/NOT DIVISIBLE

Sample Input 0

363

Sample Output 0

The number is divisible by 3

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int c=sc.nextInt();
    if(c%3==0){
      System.out.println("The number is divisible by 3");
    }else{
      int n=c%3;
      System.out.println("The number is not divisible by 3 and gives a remainder "+n);
    }
  }
}
```

C-L0-MOD2-Conditional-Challenge-5

Write a C program to check the number is either divisible by 3 or divisible by 5 and display the corresponding message

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

Hi/Hello/HiHello

Sample Input 0

1500

Sample Output 0

HiHello

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    if(n%3==0 && n%5==0){
      System.out.println("HiHello");
    }else if(n%3==0){
      System.out.println("Hi");
    }else{
      System.out.println("Hello");
    }
```

```
  }
}
```

C-L0-MOD2-Conditional-Challenge-6

Write a C program to check whether the given year is leap year or not using conditional construct

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

Leap/Not leap

Sample Input 0

1900

Sample Output 0

Not a Leap year

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int y=sc.nextInt();
```

```
      if(y%100==0 && y%400==0){
         System.out.println("Leap year");
      }else if(y%100 !=0 && y%4==0){
         System.out.println("Leap year");
      }else{
         System.out.println("Not a Leap year");
      }
   }
}
```

C-L0-MOD2-Conditional-Challenge-9

Write a program to print corresponding day based on given input

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

Print corresponding day or Invalid

Sample Input 0

0

Sample Output 0

Sunday

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
```

```
public static void main(String[] args) {
  Scanner sc=new Scanner(System.in);
  int n=sc.nextInt();
  switch(n){
    case 0:
      System.out.println("Sunday");
      break;
    case 1:
      System.out.println("Monday");
      break;
    case 2:
      System.out.println("Tuesday");
      break;
    case 3:
      System.out.println("Wednesday");
      break;
    case 4:
      System.out.println("Thursday");
      break;
    case 5:
      System.out.println("Friday");
      break;
    case 6:
      System.out.println("Saturday");
      break;
    default:
      System.out.println("Invalid");
```

```
    }
  }
}
```

C-L0-MOD2-Conditional-Challenge-10

Write a program to print the respective month name based on given input

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

Print the respective month or Invalid

Sample Input 0

3

Sample Output 0

March

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int m=sc.nextInt();
    switch(m){
```

```
case 1:
   System.out.println("January");
   break;
case 2:
   System.out.println("February");
   break;
case 3:
   System.out.println("March");
   break;
case 4:
   System.out.println("April");
   break;
case 5:
   System.out.println("May");
   break;
case 6:
   System.out.println("June");
   break;
case 7:
   System.out.println("July");
   break;
case 8:
   System.out.println("August");
   break;
case 9:
   System.out.println("September");
   break;
case 10:
```

```
            System.out.println("October");
            break;
          case 11:
            System.out.println("November");
            break;
          case 12:
            System.out.println("December");
            break;
          default:
            System.out.println("Invalid");
        }
      }
    }
```

Conditional Statement: (day-2)

Check whether a triangle:

Get 3 numbers from the user , compare the numbers and print Equilateral or Isosceles or Scalene

- If all number are same - Equilateral
- If any two numbers are same - Isosceles
- If all the numbers are different - Scalene

Hint : check the upper/lower case for the output

Input Format

Integer

Constraints

1 < n < 100

Output Format

Equilateral/Isosceles/Scalene

Sample Input 0

2 2 2

Sample Output 0

Equilateral

Explanation 0

all the numbers are same

Sample Input 1

4 5 4

Sample Output 1

Isosceles

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;
```

```
public class Solution {
   public static void main(String args[] ) throws Exception {
      Scanner sc = new Scanner(System.in);
      int a = sc.nextInt();
      int b = sc.nextInt();
      int c = sc.nextInt();
      if(a==b && b==c){
         System.out.println("Equilateral");
      }
      else if(a==b || b==c || a==c){
         System.out.println("Isosceles");
      }
      else{
         System.out.println("Scalene");
      }
   }
}
```

Electricity Bill Calculator 12:

Electricity is charged as follows:

First 100 units: ₹5 per unit

Next 100 units: ₹7 per unit

Beyond 200 units: ₹10 per unit

Given total units U, calculate and print the total bill. Use if–else if statements.

Input Format

An integer U.

Constraints

Nill

Output Format

Total bill as an integer.

Sample Input 0

50

Sample Output 0

250

Sample Input 1

125

Sample Output 1

675

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int units= sc.nextInt();
        int bill=0;
        if(units<=100){
            bill=units*5;
        }else if(units<=200){
            bill=100*5+(units-100)*7;
        }else{
            bill=100*5+100*7+(units-200)*10;
        }
        System.out.println(bill);
    }
```

}

Simple Calculator 50:

Accept two integers a, b and a character op representing an operator (+, -, *, /, %). Perform the corresponding operation and print the result. Use a switch statement. If division by zero occurs, print Error.

Input Format

a b arthimatic operator

Constraints

Nill

Output Format

Result of the operation or Error.

Sample Input 0

10

5

/

Sample Output 0

2

Sample Input 1

10

90

+

Sample Output 1

100

```
import java.io.*;

import java.util.Scanner;

import java.text.*;

import java.math.*;

import java.util.regex.*;


public class Solution {
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int a = sc.nextInt();
    int b = sc.nextInt();
    String op = sc.next();
    switch(op){
      case "+":
        System.out.println(a+b);
        break;

      case "-":
        System.out.println(a-b);
        break;

      case "*":
        System.out.println(a*b);
        break;

      case "/":
        System.out.println(a/b);
        break;

      case "%":
        System.out.println(a%b);
        break;

      default:
        System.out.println("Error");
```

```
    }
   }
}
```

Conditional Statement -1:

Write a C program to check the number is either divisible by 3 or divisible by 5 and display the corresponding message

Input Format

Accept an integer as an input

Constraints

NIL

Output Format

Hi/Hello/HiHello

Sample Input 0

1500

Sample Output 0

HiHello

Sample Input 1

93

Sample Output 1

Hi

Sample Input 2

25

Sample Output 2

Hello

```
import java.io.*;
import java.util.Scanner;
import java.text.*;
import java.math.*;
import java.util.regex.*;
```

```
public class Solution {


    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        if(a%3==0 && a%5==0){
            System.out.println("HiHello");
        }else if (a%3==0 && a%5 !=0){
            System.out.println("Hi");
        }else{
            System.out.println("Hello");
        }
    }
}
```

Bus Fare Based on Route and Time:

City buses charge different fares:

Route	Morning	Evening
A	25	30
B	20	25
C	15	20

Input Format

Route letter (A/B/C)

Time in 24-hr format integer (e.g., 9, 18).

Constraints

Nil

Output Format

Integer fare.

Sample Input 0

B

19

Sample Output 0

25

Sample Input 1

A

6

Sample Output 1

25

Sample Input 2

C

18

Sample Output 2

20

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        char c = sc.next().charAt(0);
        int t = sc.nextInt();
        switch(c){
            case 'A':
```

```
            if(t>=1 && t<=12)
               System.out.println("25");
            else
               System.out.println("30");
            break;
            case 'B':
            if(t>=1 && t<=12)
               System.out.println("20");
            else
               System.out.println("25");
            break;
            case 'C':
            if(t>=1 && t<=12)
               System.out.println("15");
            else
               System.out.println("20");
            break;
      }
   }
}
```

Character Category 1:

You are given a single character C.

Your task is to identify and print the category of the given character based on the following conditions:

If the character is an uppercase vowel (A, E, I, O, U) → print Upper Vowel

If the character is an uppercase consonant → print Upper Consonant

If the character is a lowercase vowel (a, e, i, o, u) → print Lower Vowel

If the character is a lowercase consonant → print Lower Consonant

If the character is a digit (0–9) → print Digit

Otherwise → print Special Character

Input Format

A single character C

Constraints

A single character C

Output Format

Print the category of the given character.

Sample Input 0

A

Sample Output 0

Upper Vowel

Sample Input 1

b

Sample Output 1

Lower Consonant

Sample Input 2

7

Sample Output 2

Digit

Sample Input 3

@

Sample Output 3

Special Character

```
import java.io.*;
import java.util.*;

public class Solution {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
```

```
        char c = sc.next().charAt(0);

        if (c >= 'a' && c <= 'z') {

            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
                System.out.println("Lower Vowel");
            } else {
                System.out.println("Lower Consonant");
            }

        } else if (c >= 'A' && c <= 'Z') {

            if (c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U') {
                System.out.println("Upper Vowel");
            } else {
                System.out.println("Upper Consonant");
            }

        } else if (c >= '0' && c <= '9') {

            System.out.println("Digit");

        } else {

            System.out.println("Special Character");
        }
    }
}
```

Ticket Price Calculator 1:

A theatre charges different prices based on age and day of the week.

Children (<12) : ₹100 on weekdays, ₹80 on weekends

Adults (12–59): ₹150 on weekdays, ₹120 on weekends

Seniors (60+): ₹90 on weekdays, ₹70 on weekends

Write a program to calculate the ticket price.

Input Format

First line: Integer age

Second line: String day (Monday–Sunday)

Constraints

Nil

Output Format

Single integer – the ticket price.

Sample Input 0

10

sunday

Sample Output 0

80

Sample Input 1

25

monday

Sample Output 1

150

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;
```

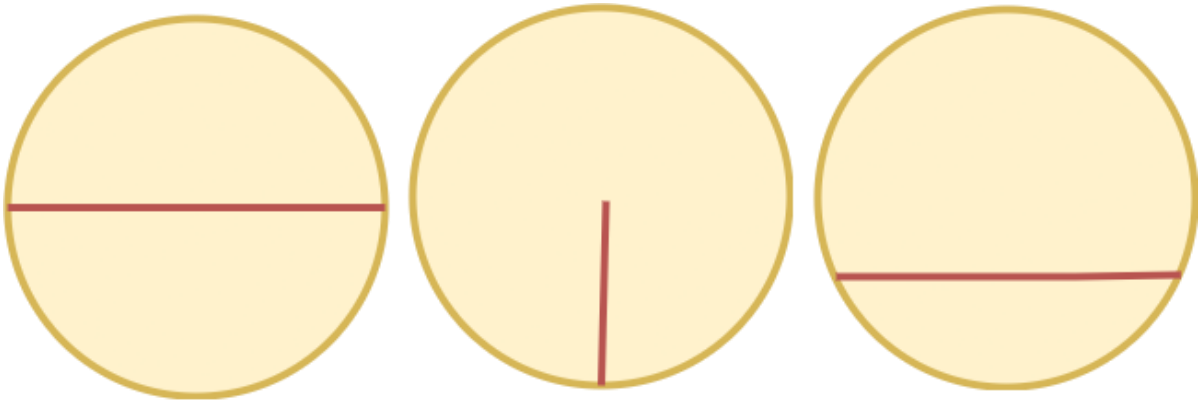
```
public class Solution {

    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        String str=sc.next();
        if(n<12){
            switch(str){
                case ("sunday"):
                    System.out.println(80);
                    break;
                case ("saturday"):
                    System.out.println(80);
                    break;
                default:
                    System.out.println(100);
                    break;
            }
        }else if(n>=12 && n<60){
            switch(str){
                case ("sunday"):
                    System.out.println(120);
                    break;
                case ("saturday"):
                    System.out.println(120);
                    break;
                default:
                    System.out.println(150);
                    break;
```

```
        }
        }else{
            switch(str){
                case ("sunday"):
                    System.out.println(70);
                    break;
                case ("saturday"):
                    System.out.println(70);
                    break;
                default:
                    System.out.println(90);
                    break;
        }
        }
    }
}
```

Minimum Cuts to Divide a Circle:

A cut that is represented by a straight line that touches two points on the edge of the circle and passes through its center, or A cut that is represented by a straight line that touches one point on the edge of the circle and its center. Some valid and invalid cuts are shown in the figures below.



Valid cut Valid cut Invalid cut

Input Format

integer

Constraints

1 <= n <= 100

Output Format

integer

Sample Input 0

4

Sample Output 0

2

Sample Input 1

3

Sample Output 1

3

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        if(n==1){
            System.out.println(0);
        return;
        }
        if(n%2==0){
```

```
            System.out.println(n/2);
        }else{
            System.out.println(n);
        }
    }
}
```

Print table - LOOPING B3

Print the multiplication table as per the multiplicant and n.

Input Format

Integer Integer

Constraints

NULL

Output Format

Integer * Integer = Integer

Sample Input 0

5 4

Sample Output 0

1 * 5 = 5
2 * 5 = 10
3 * 5 = 15
4 * 5 = 20

```
import java.io.; import java.util.; import java.text.; import java.math.; import java.util.regex.*;

public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should
be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    int t=sc.nextInt();
    for(int i=1;i<=t;i++){
        System.out.println(i+" * "+n+" = "+(n*i));
    }
}
}
```

Fibonacci Sequence 24

Display Fibonacci Sequence , from the given number The Fibonacci sequence is a sequence where the next term is the sum of the previous two terms. The first two terms of the Fibonacci sequence are 0 followed by 1.

Input Format

INT

Constraints

NULL

Output Format

N Fibonacci Sequence

Sample Input 0

10

Sample Output 0

0, 1, 1, 2, 3, 5, 8, 13, 21, 34,

Sample Input 1

8

Sample Output 1

0, 1, 1, 2, 3, 5, 8, 13,

```
import java.io.; import java.util.; import java.text.; import java.math.; import java.util.regex.*;

public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should
be named Solution. */
    Scanner sc=new Scanner(System.in);
```

```
    int n=sc.nextInt();
    int a=0,b=1;
    System.out.print("0, 1, ");
    for(int i=3;i<=n;i++){
        int c=a+b;
        System.out.print(c+", ");
        a=b;
        b=c;
    }
}
 }
```

---

factors Loop S3

Print all the factors of the given number.

Input Format

INT

Constraints

NULL

Output Format

Set of integers seperated by comma.

Sample Input 0

12

Sample Output 0

1,2,3,4,6,12

Explanation 0

Factors of 12 = 1,2,3,4,6,12

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

```
public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    for(int i=1;i<=n;i++){
        if(n%i==0){
            if(i!=n)
                System.out.print(i+",");
            else
                System.out.print(i);
        }
    }
}
 }
```

Count and Sum of Factors of a Number

Given an integer N, write a program to:

Count the total number of factors of the number.

Find the sum of all its factors.

A factor of a number is an integer that divides the number exactly without leaving a remainder.

Input Format

The first line contains an integer N.

Constraints

Nil

Output Format

Print two integers separated by a space:

First → total number of factors

Second → sum of all factors

 Explanation Factors of 6 → 1, 2, 3, 6 Count = 4 Sum = 12

Sample Input 0

6

Sample Output 0

4 12

Sample Input 1

10

Sample Output 1

4 18

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

public class Solution {

```
public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    int sum=0,count=0;
    for(int i=1;i<=n;i++){
        if(n%i==0){
            count++;
            sum+=i;
        }
    }
    System.out.println(count+" "+sum);
}
 }
```

PAD

Given a number, check whether it is an abundant or deficient or perfect number N number 6 , find the factors and add it eg 6 = 1 , 2 , 3

the given number and sum , if both are same = Perfect Number

the given number lesser than sum = Abundant Number

the given number greater than sum = Deficient Number

Input Format

Integer

Constraints

1 < n < 100000

Output Format

Abundant/Decient/Perfect Number

Sample Input 0

6

Sample Output 0

Perfect Number

Explanation 0

6 = sum of the factor 1 + 2 + 3 = 6

Sample Input 1

7

Sample Output 1

Deficient Number

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

```
public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    int sum=0;
    for(int i=1;i<=n/2;i++){
        if(n%i==0){
            sum+=i;
        }
    }
    System.out.println(sum==n?"Perfect Number":sum>n?"Abundant Number":"Deficient Number");
}
 }
```

Prime or Not 44

Check Whether a Number is Prime or Not

Input Format

INT

Constraints

1 <= n <= 10^20

Output Format

__ is a prime number. __ is not a prime number.

Sample Input 0

5

Sample Output 0

5 is a prime number.

Sample Input 1

10

Sample Output 1

10 is not a prime number.

```
import java.io.; import java.util.; import java.text.; import java.math.; import java.util.regex.*;

public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should
be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    int count=0;
    for(int i=1;i<=n;i++){
        if(n%i==0){
            count++;
        }
    }
    if(count==2)
        System.out.println(n+" is a prime number.");
    else
        System.out.println(n+" is not a prime number.");
}
 }
```

Amicable pair S3

Find whether the given number is amicable pair or not.(sum of factors of first number equals to the 2nd number as well as sum of factors of 2nd number equal to first number)

Input Format

INT INT

Constraints

NULL

Output Format

Amicable Pair or Not a Amicable Pair

Sample Input 0

220 284

Sample Output 0

Amicable Pair

Sample Input 1

123 456

Sample Output 1

Not a Amicable Pair

```
import java.io.; import java.util.; import java.text.; import java.math.; import java.util.regex.*;

public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named
Solution. */
    Scanner sc=new Scanner(System.in);
    int num1=sc.nextInt();
    int num2=sc.nextInt();
    int sum1=0,sum2=0;
    for(int i=1;i<=num1/2;i++){
        if(num1%i==0){
            sum1+=i;
        }
    }
    for(int j=1;j<=num2/2;j++){
        if(num2%j==0){
            sum2+=j;
        }
    }
    System.out.println(sum1==num2&&sum2==num1?"Amicable Pair":"Not a Amicable Pair");
}
 }
```

LCM code

Find the LCM of the given two numbers.

Input Format

Integer Integer

Constraints

null

Output Format

Integer

Sample Input 0

5 8

Sample Output 0

40

```
import java.io.; import java.util.; import java.text.; import java.math.; import java.util.regex.*;

public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should
be named Solution. */
    Scanner sc=new Scanner(System.in);
    int a=sc.nextInt();
    int b=sc.nextInt();
    int big=0,ans=0;
    if(a>b)
        big=a;
    else
        big=b;
    for(int i=1;i<=big;i++){
        if(a%i==0 && b%i==0){
```

```
            ans=i;
        }
    }
    System.out.println((a*b)/ans);
}
 }
```

---

perfect square 26

Check whether given number is perfect square or not.

Input Format

INT

Constraints

NULL

Output Format

Perfect Square / Not

Sample Input 0

10000

Sample Output 0

Perfect Square

Sample Input 1

145

Sample Output 1

Not

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

public class Solution {

```
public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    boolean b=false;
    for(int i=2;i<=n;i++){
        if(i*i==n){
            System.out.println("Perfect Square");
            b=true;
            break;
        }
    }
    if(b==false){
        System.out.println("Not");
    }
}
 }
```

Square Star Pattern

Program to Print Square Star Pattern

Input Format

Int

Constraints

NULL

Output Format

Square Star Pattern

Sample Input 0

5

Sample Output 0

```
*****
*****
*****
*****
*****
```

Sample Input 1

7

Sample Output 1

```
*******
*******
*******
*******
*******
*******
*******
```

```
import java.io.; import java.util.; import java.text.; import java.math.; import java.util.regex.*;

public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should
be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            System.out.print("*");
        }
        System.out.println();
    }
}
 }
```

Rhombus Star Pattern

Print Rhombus Star Pattern

Input Format

INT

Constraints

NULL

Output Format

Pattern

Sample Input 0

4

Sample Output 0

```
****
 ****
  ****
   ****
```

Sample Input 1

6

Sample Output 1

```
******
 ******
  ******
   ******
    ******
     ******
```

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

public class Solution {

```
public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    for(int i=0;i<n;i++){
        for(int j=0;j<i+n;j++){
            if(j<i)
                System.out.print(" ");
            else
                System.out.print("*");
        }
        System.out.println();
    }
}
 }
```

Right triangle pattern

Right triangle pattern

right angle triangle that has its perpendicular line at the right side of the triangle.

Input Format

int

Constraints

NULL

Output Format

Right angle based on the number

Sample Input 0

5

Sample Output 0

```
    *
   **
  ***
 ****
*****
```

Sample Input 1

3

Sample Output 1

```
  *
 **
***
```

```
import java.io.; import java.util.; import java.text.; import java.math.; import java.util.regex.*;

public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named
Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(i+j>=n-1)
                System.out.print("*");
            else
                System.out.print(" ");
        }
        System.out.println();
    }

}
 }
```

Pyramid Star Pattern 3

Print Pyramid Star Pattern

Input Format

INT

Constraints

NULL

Output Format

triangle shape

Sample Input 0

4

Sample Output 0

```
    *
   ***
  *****
*******
```

Sample Input 1

7

Sample Output 1

```
          *
         ***
        *****
       *******
      *********
     ***********
*************
```

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

public class Solution {

```
public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named
Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    int i,j,sp;
    for(i=1;i<=n;i++){
        for(sp=n-1;sp>=i;sp--){
            System.out.print(" ");
        }
        for(j=1;j<i*2;j++){
            System.out.print("*");
        }

        System.out.println();
    }
}
 }
```

Half Diamond Star Pattern 3

Print Half Diamond Star Pattern (odd number only will be given)

Input Format

Int (odd number)

Constraints

NULL

Output Format

pattern

Sample Input 0

5

Sample Output 0

```
*
**
***
**
*
```

Sample Input 1

7

Sample Output 1

```
*
**
***
****
***
**
*
```

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

```
public class Solution {

public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named
Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    for(int i=0;i<n;i++){
        for(int j=0;j<=i;j++){
            if(i+j>=n)
                System.out.print(" ");
            else
                System.out.print("*");
        }
        System.out.println();
    }

}
 }
```

Inverted Half Diamond Star Pattern

Print Inverted Half Diamond Star Pattern

Input Format

INT (odd number)

Constraints

NULL

Output Format

Pattern

Sample Input 0

3

Sample Output 0

```
  *
**
  *
```

Sample Input 1

5

Sample Output 1

```
   *
  **
***
  **
   *
```

import java.io.*; import java.util.*; import java.text.*; import java.math.*; import java.util.regex.*;

public class Solution {

```
public static void main(String[] args) {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named
Solution. */
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    int i,j,sp;
    for(i=0;i<=n/2;i++){
        for(sp=n/2;sp>i;sp--){
            System.out.print(" ");
        }
        for(j=0;j<=i;j++){
            System.out.print("*");
        }
        System.out.println();
    }
    for(i=0;i<n/2;i++){
        for(sp=0;sp<=i;sp++){
            System.out.print(" ");
        }
        for(j=n/2;j>i;j--){
            System.out.print("*");
        }
        System.out.println();
    }
}
```

---

1. HOLLOW SQUARE PATTERN WITH X PATTERN S6

Hollow Square Pattern with X pattern inside(Made of 0's and 1's) using functions

Input Format

INT

Constraints

NULL

Output Format

MATRIX

Sample Input 0

5

Sample Output 0

11111

11011

10101

11011

11111

Explanation 0

Box and X shape = 1

Sample Input 1

7

Sample Output 1

1111111

1100011

1010101

1001001

1010101

1100011

1111111

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int a=sc.nextInt();
        int i,j;
        for(i=1;i<=a;i++){
            for(j=1;j<=a;j++){
                if(i==1 || i==a || j==1 || j==a || j==i || (a+1==j+i)){
                    System.out.print("1");
                }else{
                    System.out.print("0");
                }
            }
            System.out.println();
        }
    }
}
```

2. HOLLOW TRIANGLE STAR PATTERN

Hollow triangle star pattern in C (nested looping)

Input Format

INT

Constraints

NULL

Output Format

Pattern : Hollow triangle star (based on the given number)

Sample Input 0

5

Sample Output 0

```
*
**
* *
*  *
*****
```

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int a=n/2+1;
        int i,j;
        for(i=1;i<=n;i++){
            for(j=1;j<=n;j++){
                if(j==1 || i==n || j==i){
                System.out.print("*");
            }else{
                System.out.print(" ");
            }
        }
        System.out.println();
        }
    }
}
```

3. HOLLOW PYRAMID STAR PATTERN 2

Print Hollow Pyramid Star Pattern

Input Format

int

Constraints

NULL

Output Format

pattern shape

Sample Input 0

5

Sample Output 0

```
    *
  *   *
 *     *
*       *
*********
```

Sample Input 1

9

Sample Output 1

```
     *
    * *
   *    *
  *      *
 *         *
*           *
*             *
*               *
*****************
```

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int a=sc.nextInt();
        int b=a*2-1;
        for(int i=1;i<=a;i++){
            for(int j=1;j<=b;j++){
                if(i==a || (j<=a && a+1==j+i) || (j>a && a-1==j-i)){
                    System.out.print("*");
                }else{
                    System.out.print(" ");
                }
            }
            System.out.println();
        }
    }
}
```

4. NUMBER WITH 1 DIAGONAL PATTERN

Pattern n = 9 98765432* 9876543*1 987654*21 98765*321 9876*4321 987*54321 98*654321 9*7654321 *87654321

Input Format

int

Constraints

NULL

Output Format

Pattern

Sample Input 0

9

Sample Output 0

98765432*

9876543*1

987654*21

98765*321

9876*4321

987*54321

98*654321

9*7654321

*87654321

Sample Input 1

8

Sample Output 1

8765432*

876543*1

87654*21

8765*321

876*4321

87*54321

8*654321

*7654321

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        /* Enter your code here. Read input from STDIN. Print output to STDOUT.
Your class should be named Solution. */
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int i,j;
        for(i=1;i<=n;i++){
            for(j=n;j>=1;j--){
                if(i==j){
                System.out.print("*");
            }else{
                System.out.print(j);
            }
        }
        System.out.println();
        }
    }
}
```

5. RECTANGLE STAR PATTERN 1

Print Rectangle Star Pattern length and width

Input Format

INT INT

Constraints

NULL

Output Format

Shape

Sample Input 0

4 6

Sample Output 0

******

******

******

******

Sample Input 1

6 4

Sample Output 1

****

****

****

****

****

****

```
import java.io.*;
import java.util.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      int a=sc.nextInt();
      int i,j;
      for(i=1;i<=n;i++){
         for(j=1;j<=a;j++){
            System.out.print("*");
      }
      System.out.println();
      }
   }
}
```

6. ZIGZAG PATTERN NUMBER

ZigZag Pattern Number

Input Format

INT

Constraints

NULL

Output Format

Pattern

Sample Input 0

4

Sample Output 0

001

002 007

003 006 008

004 005 009 010

Sample Input 1

10

Sample Output 1

001

002 019

003 018 020

004 017 021 034

005 016 022 033 035

006 015 023 032 036 045

007 014 024 031 037 044 046

008 013 025 030 038 043 047 052

009 012 026 029 039 042 048 051 053

010 011 027 028 040 041 049 050 054 055

Sample Input 2

19

Sample Output 2

001

002 037

003 036 038

004 035 039 070

005 034 040 069 071

006 033 041 068 072 099

007 032 042 067 073 098 100

008 031 043 066 074 097 101 124

009 030 044 065 075 096 102 123 125

010 029 045 064 076 095 103 122 126 145

011 028 046 063 077 094 104 121 127 144 146

012 027 047 062 078 093 105 120 128 143 147 162

013 026 048 061 079 092 106 119 129 142 148 161 163

014 025 049 060 080 091 107 118 130 141 149 160 164 175

015 024 050 059 081 090 108 117 131 140 150 159 165 174 176

016 023 051 058 082 089 109 116 132 139 151 158 166 173 177 184

017 022 052 057 083 088 110 115 133 138 152 157 167 172 178 183 185

018 021 053 056 084 087 111 114 134 137 153 156 168 171 179 182 186 189

019 020 054 055 085 086 112 113 135 136 154 155 169 170 180 181 187 188 190

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      int arr[][]=new int[n][n];
      int i,j,num=1;
      for(i=0;i<n;i++){
         if(i%2==0){
            for(j=i;j<n;j++){
               arr[j][i]=num;
               num++;
            }
         }else{
            for(j=n-1;j>=i;j--){
               arr[j][i]=num;
               num++;
            }
         }
      }
      for(i=0;i<n;i++){
         for(j=0;j<=i;j++){
            System.out.printf("%03d ",arr[i][j]);
         }
         System.out.println();
      }
   }
}
```

7. NUMBER TRIANGLE PATTERN S

Number triangle pattern n = 9

0 909 89098 7890987 678909876 56789098765 4567890987654 345678909876543 23456789098765432 1234567890987654321

Input Format

INT

Constraints

NULL

Output Format

pattern

Sample Input 0

4

Sample Output 0

0

404

34043

2340432

123404321

Sample Input 1

5

Sample Output 1

0

505

45054

3450543

234505432

12345054321

```
import java.io.*;
import java.util.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      for(int i=0;i<=n;i++){
         for(int j=n-i+1;j<=n;j++){
               System.out.print(j);
            }
         /*if(i>0){
            System.out.print(0);
         }*/
         System.out.print("0");
         for(int j=n;j>=n-i+1;j--){
            System.out.print(j);
         }
         System.out.println();
      }
   }
}
```

LEVEL - 0 NUMBER CRUNCHING

C-L0-MOD4-CHALLENGE_1

Count the number of digits in a given number

Input Format

Integer

Constraints

NUM>0

Output Format

Integer

Sample Input 0

12345

Sample Output 0

5

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      long n=sc.nextLong();
      long count=0;
      while(n>0){
         n=n/10;
         count++;
      }
      System.out.println(count);
   } }
```

C-L0-MOD4-CHALLENGE-2

Write a C program Count the number of occurrences of a digit in a given number

Input Format

The number and the digit

Constraints

NUMBER>0

Output Format

Integer

Sample Input 0

12224 2

Sample Output 0

3

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int b=sc.nextInt();
        int count=0,rem;
        while(n>0){
            rem=n%10;
            if(rem==b)
                count++;
            n=n/10;
        }
```

```
        System.out.println(count);
    }
}
```

C-L0-MOD4-CHALLENGE_3

Write a c Program to find the product of the digits in a given number

Input Format

INTEGER

Constraints

INPUT>0

Output Format

INTEGER

Sample Input 0

123

Sample Output 0

6

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int pro=1,rem;
        while(n>0){
            rem=n%10;
```

```
            pro=pro*rem;
            n=n/10;
        }
        System.out.println(pro);
    }
}
```

C-L0-MOD4-CHALLENGE_4

Print all the prime digits in a given number

Input Format

Integer

Constraints

Input>0

Output Format

Integer

Sample Input 0

12345

Sample Output 0

2 3 5

CODE :

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        long rev = 0;
        // reverse number
        while (n > 0) {
            rev = rev * 10 + n % 10;
            n = n / 10;
```

```
        }
        // check digits in correct order
        while (rev > 0) {
            long d = rev % 10;
            if (d == 2 || d == 3 || d == 5 || d == 7) {
                System.out.print(d + " ");
            }
            rev = rev / 10;
        }
    }
}
```

C-L0-MOD4-CHALLENGE_5

Write a C program to display the digits of a given number from right to left

Input Format

Integer

Constraints

Input>=1

Output Format

Integer

Sample Input 0

1234

Sample Output 0

4 3 2 1

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
```

```
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      int sum=0,rem;
      while(n>0){
         rem=n%10;
         System.out.print(rem+" ");
         sum=sum*10+rem;
         n=n/10;
      }
   }
}
```

C-L0-MOD4-CHALLENGE_6

Write a C program to find the reverse number of the given number

Input Format

Integer

Constraints

Input>0

Output Format

Integer

Sample Input 0

123

Sample Output 0

321

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      long n=sc.nextLong();
      long rem,sum=0;
      while(n>0){
         rem=n%10;
         sum=sum*10+rem;
         n=n/10;
      }
      System.out.print(sum);
   }
}
```

C-L0-MOD4-CHALLENGE_7

Write a C program to find whether the given number is Palindrome or Not

Input Format

Integer

Constraints

Input>0

Output Format

Integer

Sample Input 0

121

Sample Output 0

Palindrome

Sample Input 1

548

Sample Output 1

Not a Palindrome

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        long n=sc.nextLong();
        long rev,sum=0,temp=n;
        while(n>0){
            rev=n%10;
            sum=sum*10+rev;
            n=n/10;
        }
        if(sum==temp){
            System.out.print("Palindrome");
        }else{
            System.out.print("Not a Palindrome");
        }
    }
}
```

C-L0-MOD4-CHALLENGE_8

Find whether the given number is Adam or Not.

Input Format

Integer

Constraints

input>0

Output Format

Integer

Sample Input 0

12

Sample Output 0

Adam Number

Sample Input 1

23

Sample Output 1

Not a adam number

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int sq1=n*n;
        int sq2,rem1,rem2,sum=0,sum2=0;
        while(n>0){
            rem1=n%10;
```

```
        sum=sum*10+rem1;
        n=n/10;
    }
    sq2=sum*sum;
    while(sq2>0){
        rem2=sq2%10;
        sum2=sum2*10+rem2;
        sq2=sq2/10;
    }
    if(sq1==sum2)
        System.out.println("Adam Number");
    else
        System.out.println("Not a adam number");
  }
}
```

C-L0-MOD4-CHALLENGE_9

Find whether the given number is armstrong or not.

Input Format

Integer

Constraints

Input>0

Output Format

Integer

Sample Input 0

153

Sample Output 0

Armstrong number

Sample Input 1

157

Sample Output 1

Not an Armstrong number

CODE :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      long n=sc.nextLong();
      long sum=0,temp=n,rem;
      long digits=0;
      long t=n;
      while(t>0){
         digits++;
         t=t/10;
      }
      while(n>0){
         rem=n%10;
         sum += Math.pow(rem,digits);
         n=n/10;
      }
      if(sum==temp)
         System.out.println("Armstrong number");
      else
         System.out.println("Not an Armstrong number");
   }
}
```

C-L0-MOD4-CHALLENGE_10

Find whether the given number is strong number or not.

Input Format

Integer

Constraints

input>0

Output Format

Integer

Sample Input 0

145

Sample Output 0

Strong Number

Sample Input 1

98

Sample Output 1

Not a strong number

Sample Input 2

98

Sample Output 2

Not a strong number

Sample Input 3

98

Sample Output 3

Not a strong number

CODE :

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```
        int temp = n, sum = 0;
        while (n > 0) {
            int digit = n % 10;
            int fact = 1;
            for (int i = 1; i <= digit; i++) {
                fact = fact * i;
            }
            sum = sum + fact;
            n = n / 10;
        }
        if (sum == temp)
            System.out.println("Strong Number");
        else
            System.out.println("Not a strong number");
    }
}
```

Number Crunching Level-1

Sum of Even Digits and Product of Odd Digits

Given a number N, find the sum of all even digits and the product of all odd digits.

Input Format

Integer

Constraints

Nil

Output Format

Integer

Sample Input 0

24351

Sample Output 0

6 15

Sample Input 1

120

Sample Output 1

2 1

CODE :

```
import java.util.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      int even=0,product=1;
      while(n>0){
         int d = n%10;
         if(d%2==0){
            even=even+d;
         }else{
            product=product*d;
         }
         n=n/10;
      }
      System.out.print(even+ " "+product);
   }
}
```

Digital Pattern Sum

Given a number N, repeatedly find the sum of its digits until the result becomes a single-digit number. Output that single-digit value.

Input Format

integer 9875

Constraints

nil

Output Format

integer

2

explanition (9+8+7+5=29 → 2+9=11 → 1+1=2)

Sample Input 0

9875

Sample Output 0

2

Sample Input 1

123

Sample Output 1

6

CODE :

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        while(n >= 10){
            int sum = 0;
            while(n > 0){
                sum += n % 10;
                n = n / 10;
            }
            n = sum;
        }
        System.out.println(n);
    }
}
```

Perfect Power Digit Sum

Given a number N, find the sum of its digits each raised to the power of their position (starting from 1 on the left). Print Yes if this sum equals the number itself, otherwise No.

Input Format

Integer

89 Explanation: $8^1 + 9^2 = 8 + 81 = 89$

Constraints

nil

Output Format

Yes or No

Sample Input 0

89

Sample Output 0

Yes

Sample Input 1

135

Sample Output 1

Yes

CODE :

```
import java.util.*;
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      String s = "" + n;
      int sum = 0;
      for(int i = 0; i < s.length(); i++){
         int d = s.charAt(i) - '0';
         sum += Math.pow(d, i + 1);
      }
```

```
        if(sum == n)
            System.out.println("Yes");
        else
            System.out.println("No");
    }
}
```

Count Harshad Numbers in Range

Given two integers L and R (1 ≤ L ≤ R ≤ 10^12), count how many Harshad numbers are in the inclusive range [L, R].

A number is a Harshad (Niven) number if it is divisible by the sum of its digits.

Sum of digits = 1 + 8 = 9; 18 % 9 == 0 → Harshad.

Harshad numbers in 1..20 are: 1,2,3,4,5,6,7,8,9,10,12,18,20 (13 numbers).

Input Format

Two space-separated integers: L R

Constraints

1 ≤ L ≤ R ≤ 10^12, (R - L) ≤ 10^7 (to keep feasible per test).

Output Format

A single integer — the count of Harshad numbers in [L, R].

Sample Input 0

1 20

Sample Output 0

13

Sample Input 1

10 30

Sample Output 1

8

CODE :

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long L = sc.nextLong();
        long R = sc.nextLong();
        int count = 0;
        for(long i = L; i <= R; i++){
            long n = i, sum = 0;
            while(n > 0){
                sum = sum + (n % 10);
                n /= 10;
            }
            if(i % sum == 0)
                count++;
        }
        System.out.println(count);
    }
}
```

DATE 29/01/2026

2235. Add Two Integers

Given two integers num1 and num2, return *the sum of the two integers*.

Example 1:

Input: num1 = 12, num2 = 5

Output: 17

Explanation: num1 is 12, num2 is 5, and their sum is 12 + 5 = 17, so 17 is returned.

Example 2:

Input: num1 = -10, num2 = 4

Output: -6

Explanation: num1 + num2 = -6, so -6 is returned.

```
class Solution {
  public int sum(int num1, int num2) {
    return num1+num2;
  }
}
```

9. Palindrome Number

Given an integer x, return true *if* x *is a palindrome, and* false *otherwise*.

Example 1:

Input: x = 121

Output: true

Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: x = -121

Output: false

Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

```
class Solution {
  public boolean isPalindrome(int x) {
    int temp=x,rem,rev=0;
    while(x>0){
      rem=x%10;
      rev=rev*10+rem;
      x/=10;
    }
    if(temp==rev)
    return true;
    else
    return false;
  }
}
```

2427. Number of Common Factors

Example 1:

Input: a = 12, b = 6

Output: 4

Explanation: The common factors of 12 and 6 are 1, 2, 3, 6.

Example 2:

Input: a = 25, b = 30

Output: 2

Explanation: The common factors of 25 and 30 are 1, 5.

```
class Solution {
  public int commonFactors(int a, int b) {
    int i,k,c=1;
    if(a>b)
      k=b;
```

```
    else
      k=a;
    for(i=2;i<=k;i++){
      if(a%i==0 && b%i==0){
        c++;
      }
    }
    return c;
  }
}
```

509. Fibonacci Number

The **Fibonacci numbers**, commonly denoted F(n) form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

F(0) = 0, F(1) = 1

F(n) = F(n - 1) + F(n - 2), for n > 1.

Given n, calculate F(n).

Example 1:

Input: n = 2

Output: 1

Explanation: F(2) = F(1) + F(0) = 1 + 0 = 1.

Example 2:

Input: n = 3

Output: 2

Explanation: F(3) = F(2) + F(1) = 1 + 1 = 2.

```
class Solution {
  public int fib(int n) {
    int a[]=new int[n+1];
```

```
    if(n==0){
      return 0;
    }else if(n==1){
      return 1;
    }else{
      int i;
      a[0]=0;
      a[1]=1;
      for(i=2;i<n+1;i++){
        a[i]=a[i-1]+a[i-2];
      }
    }
    return a[n];
  }
}
```

1137. N-th Tribonacci Number

Hint

The Tribonacci sequence T_n is defined as follows:

$T_0 = 0$, $T_1 = 1$, $T_2 = 1$, and $T_{n+3} = T_n + T_{n+1} + T_{n+2}$ for n >= 0.

Given n, return the value of T_n.

Example 1:

Input: n = 4

Output: 4

Explanation:

T_3 = 0 + 1 + 1 = 2

T_4 = 1 + 1 + 2 = 4

Example 2:

Input: n = 25

Output: 1389537

```
class Solution {
  public int tribonacci(int n) {
    int a[]=new int[n+1];
    if(n==0){
      return 0;
    }else if(n==1){
      return 1;
    }else if(n==2){
      return 1;
    }else{
      int i;
      a[0]=0;
      a[1]=1;
      a[2]=1;
      for(i=3;i<=n;i++){
        a[i]=a[i-1]+a[i-2]+a[i-3];
      }
    }
    return a[n];
  }
}
```

1342. Number of Steps to Reduce a Number to Zero

Hint

Given an integer num, return *the number of steps to reduce it to zero*.

In one step, if the current number is even, you have to divide it by 2, otherwise, you have to subtract 1 from it.

Example 1:

Input: num = 14

Output: 6

Explanation:

Step 1) 14 is even; divide by 2 and obtain 7.

Step 2) 7 is odd; subtract 1 and obtain 6.

Step 3) 6 is even; divide by 2 and obtain 3.

Step 4) 3 is odd; subtract 1 and obtain 2.

Step 5) 2 is even; divide by 2 and obtain 1.

Step 6) 1 is odd; subtract 1 and obtain 0.

Example 2:

Input: num = 8

Output: 4

Explanation:

Step 1) 8 is even; divide by 2 and obtain 4.

Step 2) 4 is even; divide by 2 and obtain 2.

Step 3) 2 is even; divide by 2 and obtain 1.

Step 4) 1 is odd; subtract 1 and obtain 0.

```
class Solution {
  public int numberOfSteps(int n) {
    int c=0;
    while(n!=0){
      if(n%2==0){
        c++;
        n/=2;
      }
      if(n%2!=0){
```

```
            c++;
            n-=1;
        }
    }
    return c;
  }
}
```

258. Add Digits

Hint

Given an integer num, repeatedly add all its digits until the result has only one digit, and return it.

Example 1:

Input: num = 38

Output: 2

Explanation: The process is

38 --> 3 + 8 --> 11

11 --> 1 + 1 --> 2

Since 2 has only one digit, return it.

Example 2:

Input: num = 0

Output: 0

```
class Solution {
  public int addDigits(int num) {
    int rem;
    if(num==0){
      return 0;
    }
```

```
    rem=num%9;
    if(rem==0){
      return 9;
    }
    return rem;
  }
}
```

2651. Calculate Delayed Arrival Time

Hint

You are given a positive integer arrivalTime denoting the arrival time of a train in hours, and another positive integer delayedTime denoting the amount of delay in hours.

Return *the time when the train will arrive at the station*.

Note that the time in this problem is in 24-hours format.

Example 1:

Input: arrivalTime = 15, delayedTime = 5

Output: 20

Explanation: Arrival time of the train was 15:00 hours. It is delayed by 5 hours. Now it will reach at 15+5 = 20 (20:00 hours).

Example 2:

Input: arrivalTime = 13, delayedTime = 11

Output: 0

Explanation: Arrival time of the train was 13:00 hours. It is delayed by 11 hours. Now it will reach at 13+11=24 (Which is denoted by 00:00 in 24 hours format so return 0).

```
class Solution {
  public int findDelayedArrivalTime(int a, int d) {
    int c=a+d;
    if(c<24)
    return c;
```

```
    else
    return c-24;
  }
}
```

1281. Subtract the Product and Sum of Digits of an Integer

Hint

Given an integer number n, return the difference between the product of its digits and the sum of its digits.

Example 1:

Input: n = 234

Output: 15

Explanation:

Product of digits = 2 * 3 * 4 = 24

Sum of digits = 2 + 3 + 4 = 9

Result = 24 - 9 = 15

Example 2:

Input: n = 4421

Output: 21

Explanation:

Product of digits = 4 * 4 * 2 * 1 = 32

Sum of digits = 4 + 4 + 2 + 1 = 11

Result = 32 - 11 = 21

```
class Solution {
  public int subtractProductAndSum(int n) {
    int x,sum=0,pod=1;
    while(n>0){
      x=n%10;
```

```
        sum+=x;
        pod*=x;
        n/=10;
      }
      return pod-sum;
    }
}
```

1952. Three Divisors

Hint

Given an integer n, return true *if* n *has* ***exactly three positive divisors***. *Otherwise, return* false.

An integer m is a **divisor** of n if there exists an integer k such that n = k * m.

Example 1:

Input: n = 2

Output: false

Explantion: 2 has only two divisors: 1 and 2.

Example 2:

Input: n = 4

Output: true

Explantion: 4 has three divisors: 1, 2, and 4.

```
class Solution {
  public boolean isThree(int n) {
    int c=0,i,k=n;
    for(i=2;i<k;i++){
      if(n%i==0){
        c++;
      }
```

```
    }
    if(c==1){
      return true;
    }else{
      return false;
    }
  }
}
```

2652. Sum Multiples

Hint

Given a positive integer n, find the sum of all integers in the range [1, n] **inclusive** that are divisible by 3, 5, or 7.

Return *an integer denoting the sum of all numbers in the given range satisfying the constraint.*

Example 1:

Input: n = 7

Output: 21

Explanation: Numbers in the range [1, 7] that are divisible by 3, 5, or 7 are 3, 5, 6, 7. The sum of these numbers is 21.

Example 2:

Input: n = 10

Output: 40

Explanation: Numbers in the range [1, 10] that are divisible by 3, 5, or 7 are 3, 5, 6, 7, 9, 10. The sum of these numbers is 40.

```
class Solution {
  public int sumOfMultiples(int n) {
    int c=0,i,k=n;
    for(i=2;i<=n;i++){
```

```
        if(i%3==0 || i%5==0 || i%7==0){
            c+=i;
        }
    }
    return c;
  }
}
```

1) Nim Game

Description:

You are playing the following Nim Game with your friend:

Initially, there is a heap of stones on the table.

You and your friend will alternate taking turns, and you go first.

On each turn, the person whose turn it is will remove 1 to 3 stones from the heap.

The one who removes the last stone is the winner.

Given n, the number of stones in the heap, return true if you can win the game assuming both you and your friend play optimally, otherwise return false.

TestCase 1:

Input: n = 4

Output: false

Explanation: These are the possible outcomes:

1. You remove 1 stone. Your friend removes 3 stones, including the last stone. Your friend wins.

2. You remove 2 stones. Your friend removes 2 stones, including the last stone. Your friend wins.

3. You remove 3 stones. Your friend removes the last stone. Your friend wins.

In all outcomes, your friend wins.

Program:

```
class Solution {
    public boolean canWinNim(int n) {
        return n%4!=0;
    }
}
```

2)Divisor Game

Description:

Alice and Bob take turns playing a game, with Alice starting first.

Initially, there is a number n on the chalkboard. On each player's turn, that player makes a move consisting of:

Choosing any integer x with 0 < x < n and n % x == 0.

Replacing the number n on the chalkboard with n - x.

Also, if a player cannot make a move, they lose the game.

Return true if and only if Alice wins the game, assuming both players play optimally.

TestCase 1:

Input: n = 2

Output: true

Explanation: Alice chooses 1, and Bob has no more moves.

Program:

```
class Solution {
   public boolean divisorGame(int n) {
      return n%2==0;
   }
}
```

3) Minimum Cuts to Divide a Circle:

Description:

A valid cut in a circle can be:

A cut that is represented by a straight line that touches two points on the edge of the circle and passes through its center, or

A cut that is represented by a straight line that touches one point on the edge of the circle and its center.

TestCase1:

Input: n = 4

Output: 2

Explanation:

The above figure shows how cutting the circle twice through the middle divides it into 4 equal slices.

Program:

```
class Solution {
   public int numberOfCuts(int n) {
      if(n==1)
      return 0;
      if(n%2==0)
      return n/2;
      else
      return n;
   }
}
```

4) Count of Matches in Tournament

Description:

You are given an integer n, the number of teams in a tournament that has strange rules:

If the current number of teams is even, each team gets paired with another team. A total of n / 2 matches are played, and n / 2 teams advance to the next round.

If the current number of teams is odd, one team randomly advances in the tournament, and the rest gets paired. A total of (n - 1) / 2 matches are played, and (n - 1) / 2 + 1 teams advance to the next round.

Return the number of matches played in the tournament until a winner is decided.

TestCase1:

Input: n = 7

Output: 6

Explanation: Details of the tournament:

- 1st Round: Teams = 7, Matches = 3, and 4 teams advance.

- 2nd Round: Teams = 4, Matches = 2, and 2 teams advance.

- 3rd Round: Teams = 2, Matches = 1, and 1 team is declared the winner.

Total number of matches = 3 + 2 + 1 = 6.

Program:

```
class Solution {
   public int numberOfMatches(int n) {
      return n-1;
   }
}
```

5) Calculate Delayed Arrival Time

Description:

You are given a positive integer arrivalTime denoting the arrival time of a train in hours, and another positive integer delayedTime denoting the amount of delay in hours.

Return the time when the train will arrive at the station.

Note that the time in this problem is in 24-hours format.

TestCase1 1:

Input: arrivalTime = 15, delayedTime = 5

Output: 20

Explanation: Arrival time of the train was 15:00 hours. It is delayed by 5 hours. Now it will reach at 15+5 = 20 (20:00 hours).

Program:

```
class Solution {
   public int findDelayedArrivalTime(int arrivalTime, int delayedTime) {
        return (arrivalTime + delayedTime) % 24;
      }
   }
```

6) Smallest Even Multiple

Description:

Given a **positive** integer n, return *the smallest positive integer that is a multiple of* ***both*** 2 *and* n.

TestCase 1:

Input: n = 5

Output: 10

Explanation: The smallest multiple of both 5 and 2 is 10.

Program:

```
class Solution {
    public int smallestEvenMultiple(int n) {
        if(n%2==0)
        return n;
        else
        return n*2;
    }
}
```

7) Bulb Switcher

Description:

There are n bulbs that are initially off. You first turn on all the bulbs, then you turn off every second bulb.

On the third round, you toggle every third bulb (turning on if it's off or turning off if it's on). For the ith round, you toggle every i bulb. For the nth round, you only toggle the last bulb.

Return *the number of bulbs that are on after n rounds*.

TestCase1:

Input: n = 3

Output: 1

Explanation: At first, the three bulbs are [off, off, off].

After the first round, the three bulbs are [on, on, on].

After the second round, the three bulbs are [on, off, on].

After the third round, the three bulbs are [on, off, off].

So you should return 1 because there is only one bulb is on.

Program:

```
class Solution {
    public int bulbSwitch(int n) {
        return (int)Math.sqrt(n);
    }
}
```

9. Palindrome Number

Given an integer x, return true if x is a palindrome, and false otherwise.

Example 1:

Input: x = 121

Output: true

Explanation: 121 reads as 121 from left to right and from right to left.

Constraints:

-231 <= x <= 231 - 1

SOLUTION

```
class Solution {
   public boolean isPalindrome(int x) {
      if (x < 0) {
         return false;
      }
      String s = String.valueOf(x);

      String reversed = new StringBuilder(s).reverse().toString();

      return s.equals(reversed);
   }
}
```

1281. Subtract the Product and Sum of Digits of an Integer

Given an integer number n, return the difference between the product of its digits and the sum of its digits.

Example 1:

Input: n = 234

Output: 15

Explanation:

Product of digits = 2 * 3 * 4 = 24

Sum of digits = 2 + 3 + 4 = 9

Result = 24 - 9 = 15

Constraints:

1 <= n <= 10^5

SOLUTION

```
class Solution {
    public int subtractProductAndSum(int n) {
        int product = 1;
        int sum = 0;

        while (n > 0) {
            int digit = n % 10;
            product *= digit;
            sum += digit;
            n /= 10;
        }
        return product - sum;
    }
}
```

1342. Number of Steps to Reduce a Number to Zero

Given an integer num, return the number of steps to reduce it to zero.

In one step, if the current number is even, you have to divide it by 2, otherwise, you have to subtract 1 from it.

Example 1:

Input: num = 14

Output: 6

Explanation:

Step 1) 14 is even; divide by 2 and obtain 7.

Step 2) 7 is odd; subtract 1 and obtain 6.

Step 3) 6 is even; divide by 2 and obtain 3.

Step 4) 3 is odd; subtract 1 and obtain 2.

Step 5) 2 is even; divide by 2 and obtain 1.

Step 6) 1 is odd; subtract 1 and obtain 0.

Constraints:

0 <= num <= 106

solution

```
class Solution {
   public int numberOfSteps(int num) {
      int steps = 0;
      while(num>0){
         if(num%2==0){
            num=num/2;
       } else{
```

```
            num=num-1;
        }
        steps++;
       }
       return steps;
    }
}
```

1952. Three Divisors

Given an integer n, return true *if* n *has exactly three positive divisors. Otherwise, return* false.

An integer m is a divisor of n if there exists an integer k such that n = k * m.

Example 1:

Input: n = 2

Output: false

Explantion: 2 has only two divisors: 1 and 2.

Constraints:

- $1 <= n <= 10^4$

SOLUTION

```
class Solution {
    public boolean isThree(int n) {

        int root = (int) Math.sqrt(n);
        if (root * root != n) {
            return false;
        }

        return isPrime(root);
    }

    private boolean isPrime(int num) {
        if (num < 2) return false;
        for (int i = 2; i * i <= num; i++) {
            if (num % i == 0) return false;
        }
```

```
            return true;
        }
    }
```

2427. Number of Common Factors

Given two positive integers a and b, return *the number of* ***common*** *factors of* a *and* b.

An integer x is a **common factor** of a and b if x divides both a and b.

Example 1:

Input: a = 12, b = 6

Output: 4

Explanation: The common factors of 12 and 6 are 1, 2, 3, 6.

Constraints:

- 1 <= a, b <= 1000

Solution

```
class Solution {
    public int commonFactors(int a, int b) {
        int gcdValue = gcd(a, b);
        int count = 0;
        for (int i = 1; i <= gcdValue; i++) {
            if (gcdValue % i == 0) {
                count++;
            }
        }
        return count;
    }
    private int gcd(int x, int y) {
        while (y != 0) {
            int temp = y;
```

```
        y = x % y;
        x = temp;
    }
    return x;
  }
}
```

2652. Sum Multiples

Given a positive integer n, find the sum of all integers in the range [1, n] inclusive that are divisible by 3, 5, or 7.

Return *an integer denoting the sum of all numbers in the given range satisfying the constraint.*

Example 1:

Input: n = 7

Output: 21

Explanation: Numbers in the range [1, 7] that are divisible by 3, 5, or 7 are 3, 5, 6, 7. The sum of these numbers is 21.

SOLUTION

```
class Solution {
  public int sumOfMultiples(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
      if (i % 3 == 0 || i % 5 == 0 || i % 7 == 0) {
        sum += i;
      }
    }
    return sum;
  }
}
```

263. Ugly Number

An ugly number is a *positive* integer which does not have a prime factor other than 2, 3, and 5.

Given an integer n, return true *if* n *is an ugly number*.

Example 1:

Input: n = 6

Output: true

Explanation: 6 = 2 × 3

SOLUTION

```
class Solution {
   public boolean isUgly(int n) {
      if (n <= 0) return false;
      while (n % 2 == 0) {
         n /= 2;
      }
      while (n % 3 == 0) {
         n /= 3;
      }
      while (n % 5 == 0) {
         n /= 5;
      }
      return n == 1;
   }
}
```

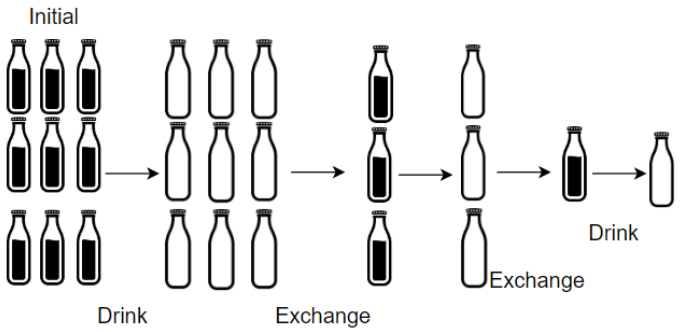
1518. Water Bottles

There are numBottles water bottles that are initially full of water. You can exchange numExchange empty water bottles from the market with one full water bottle.

The operation of drinking a full water bottle turns it into an empty bottle.

Given the two integers numBottles and numExchange, return *the maximum number of water bottles you can drink.*

Example 1:



Input: numBottles = 9, numExchange = 3

Output: 13

Explanation: You can exchange 3 empty bottles to get 1 full water bottle.

Number of water bottles you can drink: 9 + 3 + 1 = 13.

SOLUTION

```
class Solution {
    public int numWaterBottles(int numBottles, int numExchange) {
        int total = numBottles;
        int empty = numBottles;
        while (empty >= numExchange) {
            int newBottles = empty / numExchange;
            total += newBottles;
            empty = empty % numExchange + newBottles;
        }
        return total;
    }
}
```

70. Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: n = 2

Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step

2. 2 steps

SOLUTION

```
class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int first = 1;
        int second = 2;
        for (int i = 3; i <= n; i++) {
            int third = first + second;
            first = second;
            second = third;
        }
        return second;
    }
}
```

3099. Harshad Number

An integer divisible by the sum of its digits is said to be a Harshad number. You are given an integer x. Return *the sum of the digits* of x if x is a Harshad number, otherwise, return -1.

Example 1:

Input: x = 18

Output: 9

Explanation:

The sum of digits of x is 9. 18 is divisible by 9. So 18 is a Harshad number and the answer is 9.

SOLUTION

```
class Solution {
   public int sumOfTheDigitsOfHarshadNumber(int x) {
        int sum = 0;
        int temp = x;
        while( temp>0){
          sum +=temp %10;
          temp /=10;
        }
        if(x % sum==0){
          return sum;
        } else{
          return -1;
        }
   }
}
```

231. Power of Two

Given an integer n, return *true if it is a power of two. Otherwise, return false*.

An integer n is a power of two, if there exists an integer x such that n == 2^x.

Example 1:

Input: n = 1

Output: true

Explanation: $2^0 = 1$

SOLUTION

```
class Solution {
```

```
    public boolean isPowerOfTwo(int n) {
        if (n==0) return false;
        while(n%2==0){
            n=n/2;
        }
        return n==1;
    }
}
```

326. Power of Three

Given an integer n, return *true if it is a power of three. Otherwise, return false*.

An integer n is a power of three, if there exists an integer x such that n == 3^x.

Example 1:

Input: n = 27

Output: true

Explanation: 27 = 3^3

SOLUTION

```
class Solution {
    public boolean isPowerOfThree(int n) {
        if (n==0) return false;
        while(n%3==0)
        {
            n=n/3;
        }
        return n==1;
    }
}
```

342. Power of Four

Given an integer n, return *true if it is a power of four. Otherwise, return false*.

An integer n is a power of four, if there exists an integer x such that n == 4^x.

Example 1:

Input: n = 16

Output: true

Example 2:

Input: n = 5

Output: false

SOLUTION

```
class Solution {
   public boolean isPowerOfFour(int n) {
      if(n==0) return false;
      while(n%4==0){
         n=n/4;
      }
      return n==1;
   }
}
```

172. Factorial Trailing Zeroes

Given an integer n, return *the number of trailing zeroes in* n!.

Note that n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1.

Example 1:

Input: n = 3

Output: 0

Explanation: 3! = 6, no trailing zero.

Example 2:

Input: n = 5

Output: 1

Solution

```
class Solution {
```

```
    public int trailingZeroes(int n) {
        int count = 0;
        while (n > 0) {
            n /= 5;
            count += n;
        }
        return count;
    }
}
```

2119. A Number After a Double Reversal

Reversing an integer means to reverse all its digits.

- For example, reversing 2021 gives 1202. Reversing 12300 gives 321 as the leading zeros are not retained.

Given an integer num, reverse num to get reversed1, then reverse reversed1 to get reversed2. Return true *if* reversed2 *equals* num. Otherwise return false.

Example 1:

Input: num = 526

Output: true

Explanation: Reverse num to get 625, then reverse 625 to get 526, which equals num.

Solution

```
class Solution {
    public boolean isSameAfterReversals(int num) {

        if (num == 0) return true;
        if (num % 10 == 0) return false;
        return true;
    }
```

```
}
```

7. Reverse Integer

Given a signed 32-bit integer x, return x *with its digits reversed*. If reversing x causes the value to go outside the signed 32-bit integer range [-2^{31}, 2^{31} - 1], then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: x = 123

Output: 321

Example 2:

Input: x = -123

Output: -321

Solution

```
class Solution {
   public int reverse(long x) {
      long rev = 0;
      while (x != 0) {
         long digit = x % 10;
         rev = rev * 10 + digit;
         x = x / 10;
      }
         if (rev > Integer.MAX_VALUE || rev < Integer.MIN_VALUE) {
            return 0;
         }
         else
      return (int) rev;
   }
}
```

509. Fibonacci Number

The Fibonacci numbers, commonly denoted F(n) form a sequence, called the Fibonacci sequence, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

F(0) = 0, F(1) = 1

F(n) = F(n - 1) + F(n - 2), for n > 1.

Given n, calculate F(n).

Example 1:

Input: n = 2

Output: 1

Explanation: F(2) = F(1) + F(0) = 1 + 0 = 1.

Solution

```
class Solution {
    public int fib(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        int first = 0, sec = 1;
        int temp = 0;
        for (int i = 2; i <= n; i++) {
            temp = first + sec;
            first = sec;
            sec = temp;
        }
        return temp;
    }
}
```

1137. N-th Tribonacci Number

The Tribonacci sequence T_n is defined as follows:

$T_0 = 0$, $T_1 = 1$, $T_2 = 1$, and $T_{n+3} = T_n + T_{n+1} + T_{n+2}$ for n >= 0.

Given n, return the value of T_n.

Example 1:

Input: n = 4

Output: 4

Explanation:

T_3 = 0 + 1 + 1 = 2

T_4 = 1 + 1 + 2 = 4

Solution

```
class Solution {
    public int tribonacci(int n) {
        if (n == 0) return 0;
        if (n == 1 || n == 2) return 1;
        int a = 0, b = 1, c = 1;
        int temp = 0;
        for (int i = 3; i <= n; i++) {
            temp = a + b + c;
            a = b;
            b = c;
            c = temp;
        }
        return c;
    }
}
```

2739. Total Distance Traveled

A truck has two fuel tanks. You are given two integers, mainTank representing the fuel present in the main tank in liters and additionalTank representing the fuel present in the additional tank in liters.

The truck has a mileage of 10 km per liter. Whenever 5 liters of fuel get used up in the main tank, if the additional tank has at least 1 liters of fuel, 1 liters of fuel will be transferred from the additional tank to the main tank.

Return *the maximum distance which can be traveled.*

Note: Injection from the additional tank is not continuous. It happens suddenly and immediately for every 5 liters consumed.

Example 1:

Input: mainTank = 5, additionalTank = 10

Output: 60

Explanation:

After spending 5 litre of fuel, fuel remaining is (5 - 5 + 1) = 1 litre and distance traveled is 50km.

After spending another 1 litre of fuel, no fuel gets injected in the main tank and the main tank becomes empty.

Total distance traveled is 60km.

Solution

```
class Solution {
    public int distanceTraveled(int mainTank, int additionalTank) {
        int distance = 0;
        while (mainTank > 0) {
            mainTank--;
            distance += 10;
            if (distance % 50 == 0 && additionalTank > 0) {
                mainTank++;
                additionalTank--;
            }
        }
        return distance;
    }
}
```

1523. Count Odd Numbers in an Interval Range

Given two non-negative integers low and high. Return the *count of odd numbers between* low *and* high *(inclusive)*.

Example 1:

Input: low = 3, high = 7

Output: 3

Explanation: The odd numbers between 3 and 7 are [3,5,7].

Example 2:

Input: low = 8, high = 10

Output: 1

Explanation: The odd numbers between 8 and 10 are [9].

Solution

```
class Solution {
   public int countOdds(int low, int high) {
      return ((high + 1) / 2) - (low / 2);
   }
}
```

258. Add Digits

Given an integer num, repeatedly add all its digits until the result has only one digit, and return it.

Example 1:

Input: num = 38

Output: 2

Explanation: The process is

38 --> 3 + 8 --> 11

11 --> 1 + 1 --> 2

Since 2 has only one digit, return it.

Solution

```
class Solution {
    public int addDigits(int num) {
        if (num==0) return 0;
        return  (num - 1) % 9 + 1;
    }
}
```

1)Write a c program to accept the array from the user and print the array from left to right

Input Format

Number of elements followed by array as input elements size elements

Constraints

Null

Output Format

Print the array elements

Sample Input 0

10

1 2 3 4 5 6 7 8 9 10

Sample Output 0

1 2 3 4 5 6 7 8 9 10

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      int arr[]=new int[n];
      for(int i=0;i<n;i++)
      {
         arr[i]=sc.nextInt();
      }
       for(int i=0;i<n;i++)
      {
```

```
        System.out.print(arr[i]+" ");
      }
   }
```

2) Write a c program to accept the array from the user and print the array from right to left (reverse the array print)

Input Format

Number of elements followed by array as input elements

Constraints

Null

Output Format

Print the Reversal array elements

Sample Input 0

10

1 2 3 4 5 6 7 8 9 10

Sample Output 0

10 9 8 7 6 5 4 3 2 1

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      int arr[]=new int[n];
      for(int i=0;i<n;i++)
      {
```

```
            arr[i]=sc.nextInt();
        }
        for(int i=n-1;i>=0;i--)
        {
            System.out.print(arr[i]+" ");
        }
    }
}
```

3) Write a program to accept the elements into an array and display the sum of the array.

Input Format

Number of elements followed by array as input elements

Constraints

Null

Output Format

Print the sum of the array elements

Sample Input 0

10

1 2 3 4 5 6 7 8 9 10

Sample Output 0

55

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
```

```
        int n=sc.nextInt(),sum=0;
        int arr[]=new int[n];
        for(int i=0;i<n;i++)
        {
            arr[i]=sc.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            sum=sum+arr[i];
        }
        System.out.println(sum);
    }
}
```

4) Write a program to accept the array and count the number of odd and even numbers in the given array

Input Format

Number of elements followed by array as an input

Constraints

Null

Output Format

Display the number of Odd and Even numbers in the given array

Sample Input 0

10

77 67 55 44 25 14 87 95 45 92

Sample Output 0

Odd = 7

Even = 3

Explanation 0

count the odd and even in the array

print Odd count

Even count

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int arr[]=new int[n];
        int even=0,odd=0;
        for(int i=0;i<n;i++)
        {
            arr[i]=sc.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            if(arr[i]%2==0)
            {
                even++;
            }
            else
            {
                odd++;
            }
        }
```

```
        System.out.println("Odd = "+odd);
        System.out.println("Even = "+even);
    }
}
```

5) Write a program to count the number of zeros and ones in an array

Input Format

Number of elements followed by array with 0/1 as input elements

Constraints

Null

Output Format

Display the 0's and 1's count

Sample Input 0

10

0 0 0 0 0 1 1 1 1 1

Sample Output 0

zc = 5

oc = 5

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int arr[]=new int[n];
        int zero=0,one=0;
```

```
        for(int i=0;i<n;i++)
        {
            arr[i]=sc.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            if(arr[i]==0)
            {
                zero++;
            }
            else
            {
                one++;
            }
        }
        System.out.println("zc = "+zero);
         System.out.println("oc = "+one);
    }
}
```

6) Write a program to accept the array elements from the user and Find the largest element in the array.

Input Format

Number of elements Array elements

Constraints

Null

Output Format

Largest element in the array

Sample Input 0

10

10 20 30 40 50 60 70 80 90 100

Sample Output 0

100

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int arr[]=new int[n];
        int max=Integer.MIN_VALUE;
        for(int i=0;i<n;i++)
        {
            arr[i]=sc.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            if(arr[i]>max)
            {
                max=arr[i];
            }
        }
        System.out.println(max);
    }
}
```

7) Write a program to accept a array elements from the user and display the smallest element in the array...

Input Format

Number of elements Array elements

Constraints

-n < 0 < n

Output Format

Smallest element in the array

Sample Input 0

5

10 20 30 40 50

Sample Output 0

10

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int arr[]=new int[n];
        int min=Integer.MAX_VALUE;
        for(int i=0;i<n;i++)
        {
            arr[i]=sc.nextInt();
        }
```

```
        for(int i=0;i<n;i++)
        {
            if(arr[i]<min)
            {
                min=arr[i];
            }
        }
        System.out.println(min);
    }
}
```

8) You are given an array of integers. Your task is to swap alternate elements of the array. That is, swap the elements at positions:

index 0 and 1,

index 2 and 3,

index 4 and 5, and so on.

If the array has an odd number of elements, leave the last element as it is.

Input Format

n arr[0] arr[1] arr[2] ... arr[n-1] n — the number of elements in the array

arr — the array elements separated by spaces

Constraints

$1 \leq n \leq 10^5$

$-10^6 \leq arr[i] \leq 10^6$

Output Format

Print the array after performing alternate swapping.

Sample Input 0

6

1 2 3 4 5 6

Sample Output 0

2 1 4 3 6 5

Sample Input 1

5

10 20 30 40 50

Sample Output 1

20 10 40 30 50

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int arr[]=new int[n];
        int b[]=new int[n];
        int temp=0;
        for(int i=0;i<n;i++)
        {
            arr[i]=sc.nextInt();
        }
        for(int i=0;i<n-1;i=i+2)
        {
            temp=arr[i];
            arr[i]=arr[i+1];
            arr[i+1]=temp;
        }
        for(int i=0;i<n;i++)
```

```
        {
            System.out.print(arr[i]+" ");
        }
    }
}
```

9) Given the array nums consisting of 2n elements in the form [x1,x2,...,xn,y1,y2,...,yn].

Return the array in the form [x1,y1,x2,y2,...,xn,yn].

Input Format

Input: nums = [2,5,1,3,4,7], n = 3 Output: [2,3,5,4,1,7] Explanation: Since x1=2, x2=5, x3=1, y1=3, y2=4, y3=7 then the answer is [2,3,5,4,1,7].

Input: nums = [1,2,3,4,4,3,2,1], n = 4 Output: [1,4,2,3,3,2,4,1]

Input: nums = [1,1,2,2], n = 2 Output: [1,2,1,2]

Constraints

1 <= n <= 500 nums.length == 2n 1 <= nums[i] <= 10^3

Output Format

Execute the elements of Shuffled Array

Sample Input 0

6

2 4 6 8 9 1

3

Sample Output 0

2 8 4 9 6 1

Sample Input 1

10

1 2 3 4 5 6 7 8 9 10

5

Sample Output 1

1 6 2 7 3 8 4 9 5 10

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int n=sc.nextInt();
      int arr[]=new int[n];
      for(int i=0;i<n;i++)
      {
         arr[i]=sc.nextInt();
      }
      int a=sc.nextInt();
      for(int i=0;i<a;i++)
      {
         System.out.print(arr[i]+" ");
          System.out.print(arr[i+a]+" ");
         //a++;
}
}
```

10) Karthick decided to go to his friend's home. He has reached his friend's home Street. In that place, lots of buildings are there. Then he started to search for his friend's home. He forgot the door number. But he knows the address very well. Finally, he found his friend's home. He wants to know the door number of his friend's home address. Could you please tell me how did he find the door number of his friend's home?

Input Format

Input consists of number of buildings in that street and Door numbers for the each home. Karthick friend's home address.

Constraints

No Constraints

Output Format

If he found the home addrress after that returns the door number. If he not found the home,Return the value -1

Sample Input 0

5

7 9 5 3 2

5

Sample Output 0

Door Number is 002-DN

Sample Input 1

7

8 6 5 3 2 1 7

2

Sample Output 1

Door Number is 004-DN

Sample Input 2

4

8 9 6 4

0

Sample Output 2

-1

Program:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;
```

```
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int arr[]=new int[n];
        for(int i=0;i<n;i++)
        {
            arr[i]=sc.nextInt();
        }
        int k=sc.nextInt();
        for(int i=0;i<n;i++)
        {
            if(arr[i]==k)
            {
                System.out.println("Door Number is 00"+i+"-DN");
                return;
            }
        }
        System.out.println(-1);
    }
}
```

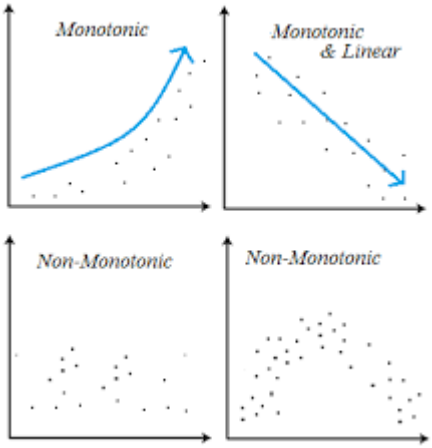
11) Write the code to Check if array is monotonic

Input Format

Given array

Constraints

Null



Output Format

Print Monotonic or Not Monotonic

Sample Input 0

7

3 4 5 6 7 8 9

Sample Output 0

Monotonic

Explanation 0

3 < 4 < 5 < 6 < 7 < 8 < 9

Sample Input 1

10

2 4 3 5 7 1 8 6 9 10

Sample Output 1

Not Monotonic

Program:

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        int n = sc.nextInt();
```

```
        int arr[] = new int[n];
        for(int i = 0; i < n; i++){
            arr[i] = sc.nextInt();
        }
        boolean increasing = true;
        boolean decreasing = true;
        for(int i = 0; i < n-1; i++){
            if(arr[i] > arr[i+1]){
                increasing = false;
            }
            if(arr[i] < arr[i+1]){
                decreasing = false;
            }
        }
        if(increasing || decreasing){
            System.out.println("Monotonic");
        } else {
            System.out.println("Not Monotonic");
        }
    }
}
```

DAY 12 (3/2/26)-HACKERRANK PROBLEMS

1. Write a program to accept a array elements from the user and display the smallest element in the array

Input Format

Accept an array elements from the user

Constraints

nil

Output Format

Display the smallest element in the array

Sample Input 0

5

10 20 30 40 50

Sample Output 0

10

Program:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;


public class Solution {


    public static void main(String[] args) {


        Scanner sc=new Scanner(System.in);

        int n=sc.nextInt();

        int a[]=new int[n],i;
```

```
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();
        }
        int min=a[0];
         for (i = 0; i < n; i++) {
            if (a[i] < min) {
                min = a[i];
            }
        }

        System.out.print(min);
    }
}
```

2. Write a program to accept the array and position in which the new element to be inserted from the user and display the array.

Input Format

number of elements

array elements

element to be inserted

position where insertion should be done

Constraints

NIL

Output Format

Updated array

Sample Input 0

10

1 2 3 4 5 6 7 8 9 10

5

2

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int i;
        int a[]=new int[n+1];
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();
        }
        int e=sc.nextInt();
        int pos=sc.nextInt();
        for(i=n;i>=pos;i--){
            a[i]=a[i-1];
        }
        a[pos-1]=e;
        for(i=0;i<n+1;i++){
        System.out.print(a[i]+" ");
        }
    }
```

}

3. Write a program to accept the array elements ,position from the user and delete the element at the given position

Input Format

number of elements

array elements

position to be deleted

Constraints

nil

Output Format

updated array

Sample Input 0

5

1 2 3 4 5

6

Sample Output 0

Deletion not possible.

Sample Input 1

10

1 2 3 4 5 6 7 8 9 10

1

Sample Output 1

2 3 4 5 6 7 8 9 10

PROGRAM:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;
```

```
public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int i;
        int a[]=new int[n];
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();
        }

        int pos=sc.nextInt();

        pos = pos - 1;

        if (pos < 0 || pos >= n) {
            System.out.print("Deletion not possible.");
        } else {
            for (i = pos; i < n - 1; i++) {
                a[i] = a[i + 1];
            }
            for (i = 0; i < n - 1; i++) {
                System.out.print(a[i] + " ");
            }
        }

    }
}
```

4. Write a program to accept the array elements from the user and find the largest element in the array.

Input Format

Number of elements

Array elements

Constraints

NIL

Output Format

Largest element in the array

Sample Input 0

10

10 20 30 40 50 60 70 80 90 100

Sample Output 0

100

PROGRAM:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;


public class Solution {


   public static void main(String[] args) {

      /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */

      Scanner sc=new Scanner(System.in);

      int n=sc.nextInt();

      int a[]=new int[n],i;

      for(i=0;i<n;i++){
```

```
            a[i]=sc.nextInt();


        }
        int max=a[0];
        for(i=0;i<n;i++){
            if(max<a[i]){
                max=a[i];
            }


        }
        System.out.print(max);


    }
}
```

5. Write a program to accept the array from the user and display all the unique elements in the array

Input Format

Accept a array size and array elements

Constraints

nil

Output Format

Display the unique element

Sample Input 0

10

1 2 2 3 3 4 4 5 5 10

Sample Output 0

1 10

PROGRAM:

```
import java.io.*;
import java.util.*;
```

```
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int i,j;
        int a[]=new int[n];
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();
        }
         boolean u =false;
           for ( i=0;i<n; i++) {
            int count=0;
            for ( j=0;j<n;j++) {
                if (a[i]==a[j]) {
                    count++;
                }
            }
            if (count==1) {
                System.out.print(a[i] + " ");
                u=true;
            }
        }
        if(!u) {
```

```
            System.out.println("No unique elements in the array");


    }
}}
```

6. Given an array of elements, write a program to find the K th smallest element in the array

Input Format

number of elements

k value

array elements

Constraints

NIL

Output Format

k th smallest element

if k value is greater than n print "Out of Range"

Sample Input 0

8

6

32 4 65 78 3 21 68 1

Sample Output 0

65

Sample Input 1

5

10

1 2 3 4 5

Sample Output 1

Out of Range

PROGRAM:

```
import java.io.*;
```

```
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
       int n = sc.nextInt();
       int k = sc.nextInt();
       int[] arr = new int[n];
       for (int i = 0; i < n; i++) {
           arr[i] = sc.nextInt();
       }
       if (k> n || k <= 0) {
           System.out.println("Out of Range");
           return;
       }
       Arrays.sort(arr);
       System.out.println(arr[k - 1]);
    }
}
```

7. Write a program to Rotate Array n times

Input Format

Accept array size,array elements and key value

Constraints

nil

Output Format

Display the rotated array

Sample Input 0

8

1 2 3 4 5 6 7 8

6

Sample Output 0

3 4 5 6 7 8 1 2

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int a[]=new int[n],i;
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();

        }
        int k=sc.nextInt();
        k=k%n;
        k=n-k;
        for (i=k;i<n;i++) { //6
            System.out.print(a[i] + " ");
        }
```

```
        for (i=0;i<k;i++) {
            System.out.print(a[i] + " ");
        }


    }
}
```

8. Write a C program to Find common elements from 3 sorted array

Input Format

Accept array elements

Constraints

nil

Output Format

common element from the sorted array

Sample Input 0

6 3 4

1 3 4 5 6 7

1 2 5

1 3 5 7

Sample Output 0

1 5

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;


public class Solution {
```

```
public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int n1=sc.nextInt();
    int n2=sc.nextInt();
    int n3=sc.nextInt();
    int a[]=new int[n1];
    int b[]=new int[n2];
    int c[]=new int[n3],i,j,k;
    for(i=0;i<n1;i++){
        a[i]=sc.nextInt();
    }
    for(i=0;i<n2;i++){
        b[i]=sc.nextInt();
    }
    for(i=0;i<n3;i++){
        c[i]=sc.nextInt();
    }
    boolean u = false;
    for(i=0;i<n1;i++){ //1
        int count = 0;
        for(j=0;j<n2;j++){//1
        for(k=0;k<n3;k++){//1 //3 //5 //7
        if(a[i]==b[j] && b[j]==c[k]){
            count++;
        }
    }
    }
    if (count == 1) {
    System.out.print(a[i] + " ");
```

```
        u = true;
        }
        }
        if (!u) {
            System.out.println("No unique elements in the array");
        }
    }
}
```

9. Write a program to accept the array and search the given element present in the array

Input Format

Accept the number of array elements,array and elements to be searched as an input

Constraints

nil

Output Format

Found/Not found

Sample Input 0

10

1 2 3 4 5 6 7 8 9 10

5

Sample Output 0

Found

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
```

```
    public static void main(String[] args) {
        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class
should be named Solution. */
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int found=0;
        int a[]=new int[n];
        int i;
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();
        }
        int k=sc.nextInt();
        for(i=0;i<n;i++){
            if(k==a[i]){
                found++;
            }
        }
        if(found==1){
            System.out.print("Found");
        }
        else{
            System.out.print("Not found");
        }
    }
}
```

10. Write a program to accept the array from the user and right rotate second half of the array by n times

Input Format

Accept the number of elements , array and number of times need to be rotated as input

Constraints

NIL

Output Format

Rotate half of the array by n times

Sample Input 0

10

1 2 3 4 5 6 7 8 9 10

4

Sample Output 0

1 2 3 4 5 10 6 7 8 9

Sample Input 1

5

1 2 3 4 5

6

Sample Output 1

1 2 3 4 5

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
```

```
        int[] a = new int[n];

        for(int i=0;i<n;i++)
            a[i] = sc.nextInt();

        int r = sc.nextInt();

        int start = (n%2==0) ? n/2 : (n+1)/2;
        int len = n - start;

        if(r < len){
            int last = a[n-1];
            for(int i=n-1;i>start;i--)
                a[i] = a[i-1];
            a[start] = last;
        }

        for(int i=0;i<n;i++)
            System.out.print(a[i] + " ");
    }
}
```

FEB 4- HACKER RANK PROBLEMS

QUESTION 1:

The Pan Am 73 flight from Bombay to New York en route to Karachi and Frankfurt was hijacked by a few Palestinian terrorists at the Karachi International Airport. The senior flight purser Neerja Banhot had to wither her fear and start evacuating the passengers on board. She pleaded the hijackers to release the oldest and the youngest person in the aircraft. Heeding to her plea the chief of the hijacker agreed to let go the oldest and the youngest. Given the ages of the passengers find the oldest and the youngest.

Input Format

The first line of input consists of an integer n, corresponding to the number of passengers in the aircraft. The next line consists of the age of passengers separated by a space.

Constraints

No Constraints

Output Format

The output prints the youngest and oldest separated by a space. Print Invalid Input if n or any one of the ages is negative.

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;


public class Solution {


    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        int n=sc.nextInt();
```

```
        int a[]=new int[n],i;
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();
        }
        int min=a[0];
        int max=a[0];
        for(i=0;i<n;i++){
            if(min>a[i]){
                min=a[i];
            }
        }for(i=0;i<n;i++){
            if(max<a[i]){
                max=a[i];
            }
        }
        System.out.println("Youngest="+min);
        System.out.println("Oldest="+max);
    }
}
```

QUESTION 2:

Given the array of integers nums, you will choose two different indices i and j of that array. Return the maximum value of (nums[i]-1)*(nums[j]-1).

Input Format

Input: nums = [3,4,5,2] Output: 12 Explanation: If you choose the indices i=1 and j=2 (indexed from 0), you will get the maximum value, that is, (nums[1]-1)(nums[2]-1) = (4-1)(5-1) = 3*4 = 12. Input: nums = [1,5,4,5] Output: 16 Explanation: Choosing the indices i=1 and j=3 (indexed from 0), you will get the maximum value of (5-1)*(5-1) = 16. Input: nums = [3,7] Output: 12

Constraints

2 <= nums.length <= 500 1 <= nums[i] <= 10^3

Output Format

Execute the Maximum Product Of Two Elements In An Array

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      int max1 = Integer.MIN_VALUE;

      int max2 = Integer.MIN_VALUE;

      for (int i = 0; i < n; i++) {

         int x = sc.nextInt();

         if (x > max1) {

            max2 = max1;

            max1 = x;

         } else if (x > max2) {
```

```
            max2 = x;
        }
    }

    int result = (max1 - 1) * (max2 - 1);
    System.out.print("Maximum product of two elements in an array is " + result);
  }
}
```

QUESTION 3:

The function accepts an integer sum and an integer array arr of size n. Implement the function to find the pair, (arr[j], arr[k]) where j!=k, Such that arr[j] and arr[k] are the least two elements of the array (arr[j] + arr[k] <= sum) and return the product of the element of this pair

Input Format

Given in the problem statement

Constraints

No Constraints

Output Format

Given in the problem statement

SOLUTION:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
```

```
import java.util.regex.*;

public class Solution {


    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int sum = sc.nextInt();

        int n = sc.nextInt();



        if (n < 2) {

            System.out.print(-1);

            return;

        }

        int min1 = Integer.MAX_VALUE;

        int min2 = Integer.MAX_VALUE;

        for (int i = 0; i < n; i++) {

            int x = sc.nextInt();

            if (x < min1) {

                min2 = min1;

                min1 = x;

            } else if (x < min2) {

                min2 = x;

            }
```

```
        }
        if (min1 + min2 <= sum) {
            System.out.print(min1 * min2);
        } else {
            System.out.print(0);
        }
    }
}
```

QUESTION 4:

A party has been organised on cruise. The party is organised for a limited time(T). The number of guests entering (E[i]) and leaving (L[i]) the party at every hour is represented as elements of the array. The task is to find the maximum number of guests present on the cruise at any given instance within T hours.

Input Format

Given in the problem statement

Constraints

1<=T<=25 0<= E[i] <=500 0<= L[i] <=500

Output Format

Given in the problem statement

SOLUTION:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
```

```
import java.util.regex.*;


public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        int[] E= new int[T];
        int[] L= new int[T];
        int i;
        for (i = 0; i<T; i++) {
            E[i] = sc.nextInt();
        }
        for (i = 0; i<T; i++) {
            L[i] = sc.nextInt();
        }
        int cu= 0;
        int max = 0;
        for (i = 0;i<T; i++) {
            cu+= E[i]-L[i];
            max= Math.max(max,cu);
        }
        System.out.println("Maximum number of guests within " + T + " hours is " + max);
```

```
}}
```

QUESTION 5:

The function accepts two positive integers ‘r’ and ‘unit’ and a positive integer array ‘arr’ of size ‘n’ as its argument ‘r’ represents the number of rats present in an area, ‘unit’ is the amount of food each rat consumes and each ith element of array ‘arr’ represents the amount of food present in ‘i+1’ house number, where 0 <= i

Input Format

Input:

r: 7 unit: 2 n: 8 arr: 2 8 3 5 7 4 1 2

Constraints

No Constrains

Output Format

Output:

4

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;

   public class Solution {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int r= sc.nextInt();
```

```
int u=sc.nextInt();

int n=sc.nextInt();

int a[]=new int[n];

int i,j,b=0,count=0;

for (i = 0; i <n; i++) {

    a[i] = sc.nextInt();

}

int req=r*u;

for (i = 0; i < n; i++) {

    b+= a[i];

    if (b>= req) {

        System.out.print("Amount of food in 1st " + (i + 1) + " houses is sufficient for all the rats");

        return;

    }

}

System.out.print(0);}}
```

QUESTION 6:

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Input Format

A list of integers representing the elements of the array.

Constraints

The length of the array (n) is at least 1. The majority element always exists in the array.

Output Format

An integer representing the majority element.

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;



public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);



        int n = sc.nextInt();

        int[] a = new int[n];



        for (int i = 0; i < n; i++) {

            a[i] = sc.nextInt();

        }
```

```
        Arrays.sort(a);

        System.out.println(a[n / 2]);

    }

}
```

QUESTION 7:

Choose an index of an array and check such that the sum of elements at previous indexes is equal to the sum of elements at next indexes from that choosen index.

Example : If Input: A[] = {-7, 1, 5, 2, -4, 3, 0} Then Output: 3., 3 is a balanced index, because: A[0] + A[1] + A[2] = A[4] + A[5] + A[6] though A[3] is the choosen one.

Input Format

read an integer n and read n number of integer elements

Constraints

Nil

Output Format

find and print the equilibrium index number or return -1 if not found

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;



public class Solution {
```

```
public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt(),i,j;
    int left=0,right=0;
    int arr[]=new int[n];
    for(i=0;i<n;i++){
        arr[i]=sc.nextInt();
    }

    for(i=0;i<n;i++){
        left=left+arr[i];
        right=0;
        for(j=i+2;j<n;j++){
          right=right+arr[j];}
            if(left==right){
               System.out.print(i+1);
                return;
            }
```

```
        }

        System.out.print(-1);

    }

}
```

QUESTION 8:

Develop a program that reads an array of integers from the user, calculates the frequency of each unique element, and prints the elements alongside their frequencies. The array's size and elements will be provided by the user at runtime.

NOTE:YOU MUST NOT USE ANY ALGORITHM AND INBUILT METHODS FOR THIS QUESTION

Input Format

The first line of input contains a single integer, N, representing the size of the array. The second line contains N space-separated integers representing the elements of the array.

Constraints

No Constraints

Output Format

The output should consist of several lines, each containing an integer and its frequency in the format element:frequency.

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;
```

```
public class Solution {


    public static void main(String[] args) {

      Scanner sc=new Scanner(System.in);

       int n=sc.nextInt(),i,j;

       int arr[]=new int[n];

       for(i=0;i<n;i++){

           arr[i]=sc.nextInt();

       }

       int count=1;

        boolean visited[]=new boolean[n];

       for(i=0;i<n;i++){

           count=1;

           if(visited[i]==false){

           for(j=i+1;j<n;j++){

               if(arr[i]==arr[j]){

                   count++;

                   visited[j]=true;

               }

           }

         System.out.println(arr[i]+":"+count);  }

       }
```

```
    }

}
```

1)
Sum of all subarrays of size K
Given an array arr[] and an integer K,
the task is to calculate the sum of all subarrays of size K.
Examples:
Input: arr[] = {1, 2, 3, 4, 5, 6}, K = 3
Output: 6 9 12 15
Explanation:
All subarrays of size k and their sum:
Subarray 1: {1, 2, 3} = 1 + 2 + 3 = 6
Subarray 2: {2, 3, 4} = 2 + 3 + 4 = 9
Subarray 3: {3, 4, 5} = 3 + 4 + 5 = 12
Subarray 4: {4, 5, 6} = 4 + 5 + 6 = 15

Coding :

```
public class Main {
public static void main(String[] args) {
int arr[] = {1, 2, 3, 4, 5, 6};
int k = 3;
for (int i = 0; i <= arr.length - k; i++) {
int sum = 0;
for (int j = i; j < i + k; j++) {
sum += arr[j];
}
System.out.print(sum + " ");
}
}
}
```

2) Maximum Sum of a Subarray with K Elements
Given an array arr[] and an integer k,
we need to calculate the maximum sum of a subarray having size exactly k.

Input : arr[] = [5, 2, -1, 0, 3], k = 3
Output : 6
Explanation : We get maximum sum by considering the subarray [5, 2 , -1]

Input : arr[] = [1, 4, 2, 10, 23, 3, 1, 0, 20], k = 4

Output : 39
Explanation : We get maximum sum by adding subarray [4, 2, 10, 23] of size 4.

CODING:

```
public class Main {

public static void main(String[] args) {

int arr[] = {5, 2, -1, 0, 3};

int k = 3;

int maxSum = Integer.MIN_VALUE;

for (int i = 0; i <= arr.length - k; i++) {

int sum = 0;

for (int j = i; j < i + k; j++) {

sum += arr[j];

}

if (sum > maxSum) {

maxSum = sum;

}

}

System.out.println(maxSum);

}

}
```

3)Given an array of integers greater than zero, find if it is possible to
split it in two subarrays (without reordering the elements),
such that the sum of the two subarrays is the same. Print the two subarrays.

Examples :

Input : Arr[] = { 1 , 2 , 3 , 4 , 5 , 5 }
Output : { 1 2 3 4 }
{ 5 5 }

Input : Arr[] = { 4, 1, 2, 3 }
Output : {4 1} {2 3}

Input : Arr[] = { 4, 3, 2, 1}
Output : Not Possible

CODING :

```
public class Main {

public static void main(String[] args) {

int arr[] = {1, 2, 3, 4, 5, 5};

int total = 0;

for (int i = 0; i < arr.length; i++) {

total += arr[i];

}

int sum = 0;

boolean found = false;

for (int i = 0; i < arr.length; i++) {

sum += arr[i];

if (sum == total - sum) {

found = true;

System.out.print("{ ");

for (int j = 0; j <= i; j++) {

System.out.print(arr[j] + " ");

}

System.out.println("}");

System.out.print("{ ");

for (int j = i + 1; j < arr.length; j++) {

System.out.print(arr[j] + " ");

}

System.out.println("}");

break;
```

```
}
}
if (!found) {
System.out.println("Not Possible");
}
}
}
```

4)Subarray of size k with given sum
Given an array arr[], an integer K and a Sum.
The task is to check if there exists any subarray
with K elements whose sum is equal to the given sum.
If any of the subarray with size K has the sum equal to
the given sum then print YES otherwise print NO.

Examples:

Input: arr[] = {1, 4, 2, 10, 2, 3, 1, 0, 20}
k = 4, sum = 18
Output: YES
Subarray = {4, 2, 10, 2}

Input: arr[] = {1, 4, 2, 10, 2, 3, 1, 0, 20}
k = 3, sum = 6
Output: YES

CODING :

```
public class Main {
public static void main(String[] args) {
int arr[] = {1, 2, 3, 4, 5, 5};
int total = 0;
for (int i = 0; i < arr.length; i++) {
total += arr[i];
}
```

```
int sum = 0;
boolean found = false;
for (int i = 0; i < arr.length; i++) {
sum += arr[i];
if (sum == total - sum) {
found = true;
System.out.print("{ ");
for (int j = 0; j <= i; j++) {
System.out.print(arr[j] + " ");
}
System.out.println("}");
System.out.print("{ ");
for (int j = i + 1; j < arr.length; j++) {
System.out.print(arr[j] + " ");
}
System.out.println("}");
break;
}
}
if (!found) {
System.out.println("Not Possible");
}
}
}
```

5)Count subarrays with all elements greater than K
Given an array of n integers and an integer k,
the task is to find the number of subarrays such that
all elements in each subarray are greater than k.

Examples:

Input: arr[] = {3, 4, 5, 6, 7, 2, 10, 11}, k= 5
Output: 6
The possible subarrays are {6}, {7}, {6, 7}, {10}, {11} and {10, 11}.

Input: arr[] = {8, 25, 10, 19, 19, 18, 20, 11, 18}, k = 13
Output: 1

CODING:

```
public class Main {

public static void main(String[] args) {

int arr[] = {3, 4, 5, 6, 7, 2, 10, 11};

int k = 5;

int count = 0;

for (int i = 0; i < arr.length; i++) {

if (arr[i] > k) {

int len = 0;

while (i < arr.length && arr[i] > k) {

len++;

i++;

}

count += (len * (len + 1)) / 2;

}

}

System.out.println(count);

}

}
```

6) Given an array and a integer k.we need to find the maximum element in each of the contiguous subarrays.
The subarrays will be [2,4,7], [4,7,1],[7,1,6] and [1,6,3].

The maximum numbers from the subarrays are
7 7 7 6

Constraints
No Constraints

Sample Input 0
6
7 8 4 5 6 2
3
Sample Output 0
8 8 6 6

CODING:

```
import java.util.Scanner;

public class Main {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

int n = sc.nextInt();

int arr[] = new int[n];

for(int i = 0; i < n; i++){

arr[i] = sc.nextInt();

}

int k = sc.nextInt();

for(int i = 0; i <= n - k; i++){

int max = arr[i];

for(int j = i; j < i + k; j++){

if(arr[j] > max){

max = arr[j];

}

}

System.out.print(max + " ");
```

```
}
}
}
```

7) Given a sequence of integers, referred to as 'nums', and another integer 'k',
the task is to find the total count of subarrays where the sum equals 'k'.
A subarray is defined as a continuous, non-empty sequence of elements within the array.

Example 1:

Input: nums = [1,1,1], k = 2
Output: 2

Example 2:

Input: nums = [1,2,3], k = 3
Output: 2

Sample Input 0
5
7 8 9 1 2
10
Sample Output 0
1

CODING:

```
import java.util.Scanner;
public class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int nums[] = new int[n];
for(int i = 0; i < n; i++){
nums[i] = sc.nextInt();
}
```

```
int k = sc.nextInt();
int count = 0;
for(int i = 0; i < n; i++){
int sum = 0;
for(int j = i; j < n; j++){
sum += nums[j];
if(sum == k){
count++;
}
}
}
System.out.println(count);
}
}
```

1) Given an array of N integers, rearrange the array in such a way that the first element is first maximum, second element is first minimum, third element is second maximum, fourth element is second minimum and so on.

Input Format:

The first line contains the value of the N integers separated by one or more spaces.

Boundary Conditions:

4 <= N <= 100

Output Format:

The N numbers alternately sorted as per the given instructions.

Example Input/Output 1:

Input:

1 2 3 4 5 6 7

Output:

7 1 6 2 5 3 4

Example Input/Output 2:

Input:

10 99 44 22 56 63

Output:

99 10 63 22 56 44

Code:

```
import java.util.*;
public class Main {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       ArrayList<Integer> list = new ArrayList<>();
       while(sc.hasNextInt()){
           list.add(sc.nextInt());
       }
       int n = list.size();
       int[] arr = new int[n];
```

```
        for(int i = 0; i < n; i++){
            arr[i] = list.get(i);
        }
        Arrays.sort(arr);
        int left = 0;
        int right = n - 1;
        while(left <= right){
            if(left != right){
                System.out.print(arr[right] + " " + arr[left] + " ");
            } else {
                System.out.print(arr[left]);
            }
            left++;
            right--;
        }
    }
}
```

2) Sort Values in Range

The program must accept N integers and sort
only the integers that are in the range (inclusive) X to Y.
Other integers (which are not within the range) positions
must not be changed. In case Y is smaller than X, consider the range (inclusive) as Y to X.

Boundary Condition(s):

2 <= N <= 100

1 <= Each integer value <= 10^5

1 <= X, Y <= 10^5

Input Format:

The first line contains N, X and Y.

The second line contains N integer values.

Example Input/Output 1:

Input:

5 99 10

56 500 25 20000 44

Output:

25 500 44 20000 56

Code:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int X = sc.nextInt();
        int Y = sc.nextInt();
        if (X > Y) {   // handle reversed range
            int temp = X;
            X = Y;
            Y = temp;
        }
        int[] arr = new int[N];
        List<Integer> list = new ArrayList<>();
        for (int i = 0; i < N; i++) {
            arr[i] = sc.nextInt();
            if (arr[i] >= X && arr[i] <= Y) {
                list.add(arr[i]);
            }
        }
        Collections.sort(list);
        int index = 0;
        for (int i = 0; i < N; i++) {
            if (arr[i] >= X && arr[i] <= Y) {
                arr[i] = list.get(index++);
            }
        }
```

```
    for (int i = 0; i < N; i++) {
      System.out.print(arr[i] + " ");
        }
    }
}
```

3) Largest Second Largest Digits Sum

The program must accept N integers and for each integer value must print the sum of the largest and the second largest digit.
If the integer value is a single digit value or does not contain at least two distinct digits, then just print the largest digit.

Boundary Condition(s):
1 <= N <= 1000
1 <= Each integer value <= 10^9
Example Input/Output 1:
Input:
5
7056 6 999 9995 800
Output:
13 6 9 14 8

Code:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        for(int i = 0; i < N; i++) {
            int num = sc.nextInt();
            int largest = -1;
            int second = -1;
```

```
            while(num > 0) {
                int digit = num % 10;
                if(digit > largest) {
                    second = largest;
                    largest = digit;
                }
                else if(digit != largest && digit > second) {
                    second = digit;
                }
                num /= 10;
            }
            if(second == -1)
                System.out.print(largest + " ");
            else
                System.out.print((largest + second) + " ");
        }
    }
}
```

4) Given an unsorted array, find the minimum difference between any pair in the given array.

Examples :

Input: {1, 5, 3, 19, 18, 25}

Output: 1

Explanation: Minimum difference is between 18 and 19

Input: {30, 5, 20, 9}

Output: 4

Explanation: Minimum difference is between 5 and 9

Input: {1, 19, -4, 31, 38, 25, 100}

Output: 5

Explanation: Minimum difference is between 1 and -4

Code:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Integer> list = new ArrayList<>();
        while(sc.hasNextInt()){
            list.add(sc.nextInt());
        }
        int n = list.size();
        int[] arr = new int[n];
        for(int i = 0; i < n; i++){
            arr[i] = list.get(i);
        }
        Arrays.sort(arr);
        int minDiff = Integer.MAX_VALUE;
        for(int i = 1; i < n; i++){
            int diff = arr[i] - arr[i-1];
            if(diff < minDiff){
                minDiff = diff;
            }
        }
        System.out.print(minDiff);
    }
}
```